Efficient Fuzzy Labeled PSI from Vector Ring-OLE

Dung Bui[1] and Kelong Cong[2]

[1] Sorbonne Université, CNRS, LIP6, Paris, France
dung.bui@lip6.fr
[2] Zama, Paris, France
kelong.cong@zama.ai

Abstract. Fuzzy-labeled private set intersection (PSI) outputs the corresponding label if there is a "fuzzy" match between two items, for example when the Hamming distance is low between the two items. Such protocols can be applied in privacy-preserving biometric authentication, proximity testing, and so on. The only fuzzy-labeled PSI protocol designed for practical purposes is by Uzun *et al.* (USENIX'21), which is based on homomorphic encryption. This design puts constraints on the item size, label size, and communication cost since it is difficult for homomorphic encryption to support large plaintext space and it is well-known that the ciphertext-expansion factor is large.

Our construction begins with a new primitive which we call vector ring-oblivious linear evaluation (vector ring-OLE). This primitive does not rely on existing instantiations of ring-OLE over the quotient ring, but leverages the more efficient vector-OLE. It is ideal for building unbalanced threshold-labeled PSI and is also of independent interest.

Our main contribution, fuzzy-labeled PSI, is bootstrapped from our threshold-labeled PSI protocol. Through a prototype implementation, we demonstrate our communication cost is up to 4.6× better than the prior state-of-the-art with comparable end-to-end latency while supporting a significantly higher label size.

1 Introduction

Private set intersection is a secure computation task where two parties compute the intersection of their private sets without revealing anything beyond the result. It has many real-world uses, such as advertising [IKN+19], contact discovery [KRS+19], and password breach detection [CHLR18, CMdG+21]. Over the years, significant progress has been made [HFH99, FNP04, PSSW09, PRTY20, RR22], especially following the introduction of pseudorandom correlation generators [BCG+19b, BCG+20], which enabled more efficient protocols [RS21, RR22, CILO22, BC23]. Variants of PSI exist, such as PSI with labels, where a label is revealed when two items match. In this work, we are interested in a variant of labeled PSI that is *fuzzy-labeled PSI* based on PCG.

Specifically, we consider an unbalanced setting, i.e., the receiver has one item and the sender has many items (possibly millions). This problem is also known as private membership testing. Two items are matched if they are close in some distance metric, e.g., Hamming distance. Labeled means a label is returned if two items "fuzzily" match, instead of a binary flag that indicates that a match is found. This functionality is especially useful in applications such as biometric authentication, where the only thing we may want to check is if a specific fingerprint (or face) matches a fingerprint (or face) from a database [UCK+21]. Another application is privacy-preserving proximity-testing [HOS17], to compute the output whether two parties are close enough without revealing their relative distances or positions. The only labeled fuzzy PSI protocol in Hamming distance for high dimensions (e.g., $d = 128$) is by Uzun *et al.* [UCK+21], using sub-sampling and a t-out-of-T threshold PSI over leveled homomorphic encryption (HE). It achieves communication independent of the sender's set size, but suffers from large ciphertext expansion [CDPP22] and limited label size support. Chaudhuri *et al.* [CFR23] proposed fuzzy PSI (without label) using additive homomorphic encryption (AHE), which is slower asymptotically but outperforms [UCK+21] concretely for databases up to 1M items.

Another line of work by Garimella, Rosulek, and Singh [GRS22, GRS23] initiated a structure-aware PSI (without labels) where the receiver holds N balls of radius σ and dimension d and the sender holds a set of M points. In the end, the receiver learns which of the sender's points are inside one of their balls. These constructions use Function Secret Sharing (FSS) [BGI15] and Oblivious Transfer (OT) for L_∞ and L_1 distances, with recent work [GGM24] improving computation via incremental FSS. Van Baarsen and Pu [vBP24] constructed a fuzzy PSI based on the decisional Diffie-Hellman

assumption (DDH) for general L_p distances, but with high cost for large d. Recently, [GQL+24] introduced Fuzzy mapping, a new primitive supporting Hamming distance with linear complexity based on AHE and DDH. The details of related works can be found in Section 1.2.

1.1 Our Contribution

Vector ring-OLE. We propose a new primitive called vector ring-oblivious linear function evaluation (vector ring-OLE) in Section 4 where we achieve a batch of m instances of ring-OLE of degree d ($m \gg d$). This primitive is used to construct our fuzzy-labeled PSI protocols and it may be of independent interest. Using the chosen ring-OLE in previous works [GN19, CILO22], it is possible to obtain vector ring-OLE correlations of length m of degree d over a field $\mathbb{F}$ from $O(md)$ random OLEs. But the communication cost is at least $6md \log |\mathbb{F}|$ (online and offline cost) [CILO22]. We compare with [GN19, CILO22] in the same setting and present the results in Table 1. Compared to the state-of-art of chosen ring-OLE [GN19, CILO22], our vector ring-OLE enjoys two main advantages:

1. Our protocol achieves the same functionality, for the same parameters with a communication cost that is over 50% smaller compared to prior work [GN19, CILO22].
2. Our protocol has a faster setup phase using vector-OLE (VOLE) while prior work needs OLE [GN19, CILO22]. When we cast their constructions to follow our functionality, [GN19] needs $4md$ OLEs and it is instantiated from homomorphic encryption. Chongchitmate *et al.* [CILO22] need $2md$ OLEs, which is instantiated from ring-OLE. The setup cost of VOLE setup is much faster (less than a second [WYKW21]) when compared to ring OLE or OLE (10–20 seconds [BCG+20]).

Fuzzy-labeled PSI. Our primary contribution is a fuzzy-labeled PSI protocol (Section 5) that is built on top of our threshold-labeled PSI protocol (Section 5.1). Our fuzzy-labeled PSI improves over the state of the art and other related works [UCK+21, CILO22, GRS22, CFR23, vBP24, GGM24] in the following aspects:

- Our fuzzy-labeled PSI focuses on the Hamming distance and supports a high dimension (up to κ bits) efficiently as opposed to [GRS22, vBP24, GGM24], which only support less than 10 dimensions efficiently.
- The asymptotic online communication complexity does not depend on the computational security parameter κ.
- We support a label space that is not fixed in bit-length size and can go up to κ bits, which was not possible in prior works [GRS22, CILO22, CFR23, GGM24, GQL+24]. The works of [vBP24, GQL+24] also support labels as an extension of their constructions; however, labeled PSI in [vBP24] relies on ElGamal encryption, whereas [GQL+24] uses both ElGamal and AHE, resulting in poor performance.
- We implemented our protocol and it outperforms the state-of-the-art [UCK+21] for database sizes of up to 1 million in both computation and communication. For smaller database sizes, we achieve 4.6× improvement in communication.

1.2 Related Work

PCG-based PSI. At the time of writing, PCG-based PSI protocols [RS21, RR22, CILO22, BC23] are the top-performing PSI protocols in terms of both communication and computation costs. The idea of using PCG to improve PSI was first introduced by Rindal and Schoppmann [RS21], which combines oblivious key value store (OKVS) and vector-OLE (a concrete type of PCG). Later, two concurrent works [RR22, BC23] introduced various optimizations (subfield) vector-OLE instead of vector-OLE to reduce the communication cost. While [RR22] uses the same framework of [RS21], the authors designed a better construction for OKVS, giving a reduction of encoding parameter from $1.3n$ to $1.23n$. The work of Bui and Couteau [BC23] designed a new data structure called the generalized Cuckoo hashing (more than one item per bin), this data structure can be plugged into several hashing techniques for better efficiency or security.

Threshold PSI. In our work, we focus on a variation of PSI called threshold PSI. Several works consider this problem but all of the constructions have communication cost of at least linear in set size, and they are secure in the semi-honest setting, or in the malicious setting by relying on heavy public key cryptography. The work of Ghosh and Simkin [GS19] was the first to investigate a theoretical lower bound of threshold PSI. The authors show that any protocol has to have a communication complexity of at least $\Omega(t)$ where t is the maximum allowed size of the symmetric set difference. They also propose two protocols for threshold PSI against a malicious setting that have the asymptotic communication complexity of $\tilde{O}(t)$ and $\tilde{O}(t^2)$ based on fully homomorphic encryption and additively homomorphic encryption, respectively. A variant of threshold PSI is threshold-labeled PSI (TLPSI) where parties only receive the label associated with the item in the intersection set if and only if the size of the intersection is larger than the threshold parameter. All practical TLPSI instantiations either use garbled circuits and/or homomorphic encryption [UCK+21], or are only effective for a specific threshold parameter [ZCL21].

Unbalanced fuzzy PSI. We consider a variant of unbalanced fuzzy PSI in Hamming distance where the dimension d is as large as 128, the only efficient fuzzy-labeled PSI protocol is given by Uzun *et al.* [UCK+21] Their protocol is constructed using a sub-sampling procedure and a t-out-of-T threshold-labeled PSI based on a leveled homomorphic encryption (HE) scheme. Suppose the database items are bit-strings, the sub-sampling procedure takes T sub-samples of a much smaller bit-length of every database item such that if two items are (Hamming distance) close, then some threshold t of the sub-samples will match. As such, the t-out-of-T threshold-labeled PSI takes input the sub-samples and outputs the corresponding label if more than t-out-of-T of the sub-samples match. The advantage of using a HE-based PSI is that the communication complexity does not depend on the larger (sender's) set. But the ciphertext expansion factor is much larger in HE when compared to traditional encryption schemes ($20,000\times$ of the original plaintext in some cases [CDPP22]), so the communication advantage of HE only becomes apparent for very large databases. Additionally, parameters for HE must be selected to balance functionality and efficiency, which makes it challenging to support labels with larger bit-length.

Another recent work designed a fuzzy PSI protocol (without labels) using OLE and additive homomorphic encryption (AHE) [CFR23] where the authors use the same sub-sampling technique as [UCK+21] but the t-out-of-T threshold PSI is based on a set reconciliation protocol [MTZ03]. Asymptotically the communication cost is $O(m)$ where m is the database size, which is asymptotically worse than that of [UCK+21]. But concrete performance shows that it outperforms [UCK+21] for a database size of up to a million.

A new line of work by Garimella, Rosulek, and Singh [GRS22, GRS23] proposed a structure-aware PSI (without labels) where the receiver holds N balls of radius σ and dimension d and the sender holds a set of M points. In the end, the receiver learns which of the sender's points inside one of their balls. Their construction is based on weak function secret sharing (FSS) and obvious transfer (OT). Malicious security was achieved in a follow up work by ensuring multi FSS encodes the same set using the cut-and-choose approach [GRS23]. Their construction only supports L_∞ and L_1 distances and the receiver's computation remains substantial, proportional to the cardinality N of the receiver's input balls. [GGM24] overcame the computational limitation by introducing a new incremental FSS that leads to an OT-based construction where the computation only proportion to the description size of the receiver's ball. Additionally, Van Baarsen and Pu [vBP24] proposed a new fuzzy PSI from ElGamal encryption based on the decisional Diffie-Hellman assumption (DDH) that supports L_∞ and L_p for $p \in [1, \infty)$ distances. For a given distance such as L_1 (Hamming distance), their communication and computation costs are still proportional to $(2^d \cdot N)$, leading to poor performance if the dimension is high (larger than 10) and N is large. Recently, [GQL+24] proposed a construction that supports Hamming distance by initiating a new primitive called fuzzy mapping (Fmap), which can map each point of two parties to a set of identifiers if they are closed. The construction achieves strictly linear complexity with input size relying on the AHE and DDH assumption.

2 Technical Overview

2.1 Vector Ring-OLE from VOLE

Construction. Our goal is to build correlations of the form $(z_i = u_0 \cdot u_i + v_i)_{i\in[m]}$ where $\deg(u_0) = \deg(u_i) = d$ for $d \ll m$, and $\deg(z_i) = \deg(v_i) = 2d$ for all $i \in [m]$. The receiver inputs u_0 and gets $(z_i)_{i\in[m]}$ while sender inputs $(v_i)_{i\in[m]}$ and gets $(u_i)_{i\in[m]}$. The correlation is available from VOLE.

Given two polynomials of degree d, their product can be computed by evaluating on $2d+1$ fixed evaluation points $\{x_1, \ldots, x_{2d+1}\}$, perform point-wise multiplication and then interpolate the $2d+1$ points to obtain a degree $2d$ polynomial. In other words, $(z_i(x_1), z_i(x_2), \ldots, z_i(x_{2d+1}))$ and $(v_i(x_1), v_i(x_2), \ldots, v_i(x_{2d+1}))$ are additive shares of

$$(u_0(x_1), u_0(x_2), \ldots, u_0(x_{2d+1})) \odot (u_i(x_1), u_i(x_2), \ldots, u_i(x_{2d+1})),\ i \in [m]$$

where $\odot$ denotes element-wise multiplication. Since z_i and v_i are degree-$2d$ polynomials, if parties hold these shares then z_i and v_i can be constructed. If we view the problem from another angle by fixing the evaluation point x_j, we have $(z_1(x_j), z_2(x_j), \ldots, z_m(x_j))$ and $(v_1(x_j), v_2(x_j), \ldots, v_m(x_j))$ are shares of $u_0(x_j) \cdot (u_1(x_j), u_2(x_j), \ldots, u_m(x_j))$ for $j \in [2d+1]$. Therefore if we define $\mathbf{u_0} := (u_0(x_j))_{j\in[2d+1]}$, matrices $\mathbf{U}, \mathbf{V}, \mathbf{Z} \in \mathbb{F}^{m\times(2d+1)}$ as the column $\mathbf{U}_j := \{u_1(x_j), u_2(x_j), \ldots, u_m(x_j)\}$, and $\mathbf{V}_i, \mathbf{Z}_i$ are defined the same way as $\mathbf{U}_i$ from the polynomials v_i and z_i, then the goal is to securely distributed the share $\mathbf{V}$ and $\mathbf{Z}$ of $\mathbf{u_0} \odot \mathbf{U}$ to parties, where $\mathbf{u}_0$ and $\mathbf{V}$ are chosen in advance by receiver and sender. This kind of functionality can be achieved using VOLE correlations as the masking for the parties' input. The outline is given below.

Step 1. First of all, we use VOLE of length m and execute it $2d+1$ times to give sender $\mathbf{U}', \mathbf{V}' \leftarrow\$ \mathbb{F}^{m\times(2d+1)}$, and give receiver vector $\mathbf{u_0'} \leftarrow\$ \mathbb{F}^{2d+1}, \mathbf{Z}' \in \mathbb{F}^{m\times(2d+1)}$ such that $\mathbf{Z}' := \mathbf{u_0'} \odot \mathbf{U}' + \mathbf{V}'$.

Step 2. We now de-randomize this correlation to get $\mathbf{Z} = \mathbf{u_0} \odot \mathbf{U} + \mathbf{V}$, the naive method to do de-randomize is:

1. Receiver sends $\Delta\mathbf{u}_0 = \mathbf{u}_0 - \mathbf{u}_0' \in \mathbb{F}^{2d+1}$ to sender.
2. Sender then sends $\Delta\mathbf{U} = \mathbf{U} - \mathbf{U}' \in \mathbb{F}^{m\times(2d+1)}$ and $\Delta\mathbf{V} = \Delta\mathbf{u}_0 \odot \mathbf{U}' + \mathbf{V} - \mathbf{V}' \in \mathbb{F}^{m\times(2d+1)}$ to receiver.
3. Now receiver can locally compute $\mathbf{Z} := \mathbf{u}_0 \odot \Delta\mathbf{U} + \Delta\mathbf{V} + \mathbf{Z}'$, by a simple calculation.

Step 3. Viewing this relation from each row i of matrix $\mathbf{Z}$, parties now can get a share of $(u_0 \cdot u_i)$ in the form of evaluation values for $i \in [m]$. In other words, row i^{th} of $\mathbf{Z}$ defines a polynomial z_i of degree $2d$ such that $z_i = u_0 \cdot u_i + v_i$.

Optimization. Step 2 above requires $2m(2d+1)|\mathbb{F}|$ bits of communication. We reduce both communication and computation costs by 25% using the observation that the sender does not need to de-randomize the whole $\mathbf{U}'$, sender can reuse the first $(d+1)$ columns of $\mathbf{U}'$ (denote as $\mathbf{U}'[1, d+1]$) to define the set of degree-d polynomial $(u_i)_{i\in[m]}$ by interpolation each row $i \in [m]$ of $\mathbf{U}'[1, d+1]$. Therefore, sender only needs to send the second half of $\Delta\mathbf{U}$, and the computation of receiver to reconstruct $\mathbf{Z}$ is reduced by a half also. On the receiver side, the first $(d+1)$ columns $\mathbf{Z}$ is obtained by only adding $(d+1)$ columns of $\Delta\mathbf{V}$ and $\mathbf{Z}'$ (since this part is already de-randomized) and the last d columns is obtained in the same way as Step 2.

2.2 Fuzzy-Labeled PSI

Our fuzzy-labeled PSI (FLPSI) construction is based on two main building blocks (1) an efficient threshold-labeled PSI (TLPSI) for small threshold parameters and (2) a masked OPRF to transform any TLPSI into a fuzzy-labeled PSI while keeping a low false negative rate (FNR) and false positive rate (FPR).

Threshold-labeled PSI. The threshold labeled PSI (TLPSI) functionality takes input X from the receiver and Y and ℓ from the sender, and then outputs ℓ and $|X \cap Y|$ to the receiver if $|X \cap Y| \geq t$ where t is some threshold. This functionality can be instantiated using Shamir secret sharing and our vector ring-OLE functionality. Our construction is only efficient for small threshold parameter t however we show that small t and d values are enough for us to construct an efficient fuzzy-labeled PSI. The details are given in Section 5.1.

Fuzzy-labeled PSI. In our setting, receiver has an input $x \in \{0,1\}^d$ while sender has a set of inputs $Y = \{(y_i, l_i)\}$. If x and y_i are close enough in the Hamming distance $\mathsf{HW}(x \oplus y) \leq d_k$ for a threshold $d_k \ll d$ then receiver learns the corresponding label l_i. We now show how to transform from our threshold-labeled PSI to fuzzy-labeled PSI. Our transformation is inspired by [UCK+21] which uses sub-sampling to gain efficiency and masked OPRF to protect privacy.

In particular, the sub-sampling allows us to map receiver and sender's input x and y_i into sets $\bar{X}$ and $\bar{Y_i}$ of size T, respectively, such that if $\mathsf{HW}(x \oplus y) \leq d_k$ then $|\bar{X} \cap \bar{Y_i}| \geq t$ for a small t. The sets $\bar{X}$ and $\bar{Y_i}$ are constructed by applying x and y_i with T different masks $\{m_i\}_{i\in[T]}$, such that $\mathsf{HW}(m_i) = d_s \leq d$. Namely, $\bar{X} \leftarrow \{x \wedge m_1, \ldots, x \wedge m_T\}$ and $\bar{Y_i} \leftarrow \{y \wedge m_1, \ldots, y \wedge m_T\}$. For $d_k \ll d$ and an appropriate parameters choice for d_s, T we only need a very small t ($t = 2$ in our instantiation) with low false negative rate and false positive rate (FNR and FPR), details of the parameter choices are in Section 6.2.

Observe that fuzzy-labeled PSI is instantiated using a threshold PSI with a small threshold parameter. Receiver and sender now can join the threshold-labeled PSI for a threshold parameter t with input $\bar{X}$ and $\{\bar{Y_i}\}_{i\in[m]}$, after that receiver learns label l_i and $\bar{X} \cap \bar{Y_i}$ if $|\bar{X} \cap \bar{Y_i}| \geq t$. Since our threshold-labeled PSI is instantiated from our vector ring-OLE, to instantiate m TLPSI of input size T it requires only one $\mathcal{F}_{\mathsf{vrOLE}}^{m,T}$.

If sub-sampling masks $\{m_i\}_{i\in[T]}$ are public, this protocol leaks the d_s sub-sampled plaintext bits of sender's input $y_i \in \{0,1\}^d$ if there is an intersection above the threshold t. This leakage allows receiver to learn information about sender's input, for example, when in case of a false-positive match, receiver will now learn with confidence the positions in the bit vector of sender's input. To resolve this concern, a masked OPRF protocol is used where receiver inputs her item x and sender inputs a PRF key k and T masks. At the end of the protocol, receiver obtains $\{\mathsf{PRF}_k(x \wedge m_1), \mathsf{PRF}_k(x \wedge m_2), \ldots, \mathsf{PRF}_k(x \wedge m_T)\}$. This functionality is very similar to the OPRF functionality except that a mask is applied to input of receiver prior to the OPRF, it is instantiated using garbled circuits [Yao86] where the cost is minor and only dependent on T, and not m. Finally, the two parties execute the threshold-labeled PSI functionality using the masked OPRF output as the input to obtain the label. For security, our fuzzy-labeled PSI is secure against semi-honest setting and follows from the security of TLPSI and masked OPRF.

3 Preliminaries

Below we describe the notation and some important ideal functionalities.

3.1 Notation

The notation $x \leftarrow\!\$\ \mathcal{D}$ denotes sampling from a probability distribution $\mathcal{D}$, with uniform sampling if $\mathcal{D}$ is a set. A sequence from a to b (inclusive) is written as $[a, b]$, and $[b]$ implies the sequence starts at 1. We use κ and λ for computational and statistical security parameters, and denote an adversary as $\mathcal{A}$.

For a matrix $\mathbf{M} \in \mathbb{F}^{n\times m}$, $\mathbf{M}_{i,j}$ refers to the entry in the i^{th} row and j^{th} column; $\mathbf{M_j}$ and $\mathbf{M^i}$ refer to the j^{th} column and i^{th} row, respectively. $\mathbf{M}[i, j]$ is the submatrix from the i^{th} to j^{th} column. The symbol $\odot$ represents the point-wise product between a vector and a matrix: for $\mathbf{u} = (u_1, \ldots, u_m) \in \mathbb{F}^m$, $\mathbf{u} \odot \mathbf{M} = (u_1 \cdot \mathbf{M_1}, \ldots, u_m \cdot \mathbf{M_m}) \in \mathbb{F}^{n\times m}$. $0^d \| \mathbf{M} \in \mathbb{F}^{n\times(d+m)}$ denotes a matrix with the first d columns as zeros followed by $\mathbf{M}$. For a polynomial $f(x) \in \mathcal{R}$, $\deg(f)$ is the degree of $f(x)$.

For two vectors $\mathbf{u}, \mathbf{v} \in \mathbb{F}^n$, $\langle \mathbf{u}, \mathbf{v} \rangle$ is the inner product operation.

For a polynomial $f(x) \in \mathcal{R}$, we denote $\deg(f)$ as the degree of $f(x)$. We use κ and λ for computational and statistical security parameters, respectively. We denote an adversary as $\mathcal{A}$. In a two-party protocol, offline communication cost is the number of bits exchanged in messages independent of

secret inputs, while online communication cost refers to bits exchanged in messages dependent on the secret inputs.

3.2 Secure Two-Party Computation

In our work, all proofs are given in simulation-based proof and we give the formal definition of security under the semi-honest setting below.

Semi-honest security. Given a security parameter κ and a pair of inputs $(X, Y) \in \{0,1\}^*$, let $\text{view}_1^{\Pi}(X, Y, \kappa)$ and $\text{view}_2^{\Pi}(X, Y, \kappa)$ be the views of P_1 and P_2 respectively in the protocol Π, $\mathsf{out}^{\Pi}(X, Y, \kappa)$ be the output of the protocol, and $f_i(X, Y)$ be the output of P_i from the ideal functionality $f := (f_1, f_2)$. The protocol Π securely computes f in the presence of static semi-honest adversaries if there exist PPT simulators Sim_1 and Sim_2 such that for all valid inputs X, Y of f,

$$\big(\text{view}_1^{\Pi}(X, Y, \kappa), \mathsf{out}^{\Pi}(X, Y, \kappa)\big) \approx (\mathsf{Sim}_1(1^{\kappa}, X, f_1(X, Y)), f(X, Y))\,;$$
$$\big(\text{view}_2^{\Pi}(X, Y, \kappa), \mathsf{out}^{\Pi}(X, Y, \kappa)\big) \approx (\mathsf{Sim}_2(1^{\kappa}, Y, f_2(X, Y)), f(X, Y))\,.$$

3.3 Threshold Shamir's Secret Sharing

Shamir's secret sharing allows each of d parties to get a share of the secret input such that this secret can only be revealed by at least t honest parties. The functionality is given in Figure 6.

Fig. 1: $\mathcal{F}_{\mathsf{TSSS}}^{d,t}$ over $\mathbb{F}$.

PARAMETERS: Given a field $\mathbb{F}$, a threshold value t, and d distinct points $t_i \in \mathbb{F}$ such that $t_i \neq 0$.

FUNCTIONALITY:

- **Share.** For a secret input $s \in \mathbb{F}$, pick a random polynomial f of degree $(t-1)$ over $\mathbb{F}[x]$ such that $f(0) = s$. Output the set $S := \{f(t_i)\}_{i \in [d]}$.
- **Reconstruct.** Given a vector $\mathbf{c} := \{c_i\}_{i \in [d]}$, if there are at least t value c_i such that $c_i = f(t_i)$ for $i \in [d]$, reconstruct f and output $s := f(0)$. Otherwise, output $\perp$.

3.4 Ideal Functionality of Fuzzy-Labeled PSI

The ideal functionality of fuzzy-labeled PSI in the semi-honest setting is shown in Figure 2. Our fuzzy-labeled PSI is constructed from a threshold-labeled PSI, we present its ideal functionality on Figure 3. Since fuzzy matching is not exact and may have false positives or false negatives due to techniques such as locality-sensitive hashing and the sub-sampling method used in [UCK+21], and the permute and partition method used in [CFR23], which we define the context as below.

Definition 1 (Closeness Domain [CFR23]). *We say that $\delta := (\mathcal{D}, \mathsf{CI})$ is a closeness function that takes any $x, y \in \mathcal{D}$ and outputs a member of $\{\mathsf{close}, \mathsf{far}\}$, so that CI is symmetric (i.e., $\mathsf{CI}(x, y) = \mathsf{CI}(y, x)$).*

Definition 2 (Correctness of Fuzzy-Labeled PSI (FLPSI) [CFR23]). *Π_{FLPSI} protocol is defined for a closeness domain $(\mathcal{D}, \mathsf{CI})$ and a label space $\mathcal{LS}$ and for two parties: a sender and a receiver. The receiver inputs a query $x \in \mathcal{D}$, and the sender inputs a database $Y := \{(y_i, \ell_i)\}_{i \leq m}$ where items $y_i \in \mathcal{D}$ and labels $\ell_i \in \mathcal{LS}$. At the end, the receiver receives a set R, and the sender receives $\perp$. The set R is defined in the following correctness.*

***Correctness**. We denote by TPR the true positive rate and by TNR the true negative rate. For x and Y, the output set R consists of labels $\ell \in \mathcal{LS}$ such that for each $i \in [m]$:*

1. *If $\mathsf{CI}(x, y_i) = \mathsf{far}$, then $\Pr[\ell_i \notin R] \geq \mathsf{TNR}$,*
2. *If $\mathsf{CI}(x, y_i) = \mathsf{close}$, then $\Pr[\ell_i \in R] \geq \mathsf{TPR}$.*

In our work, we define CI as the Hamming distance with a threshold rate t, i.e., with overwhelming probability, $\mathsf{CI}(x, y_i) = \mathsf{close} \iff \mathsf{HW}(x \oplus y_i) \leq t$, otherwise, CI outputs far, where HW denotes the Hamming weight. Additionally, the false positive rate FPR is $1 - \mathsf{TPR}$ and the false negative rate FNR is $1 - \mathsf{TNR}$.

We note that in our construction, the leakage profile of the ideal FLPSI functionality depends on the real Π_{FLPSI} protocol, the challenge we are facing is that ideal functionality must align precisely with the behavior of the real-world protocol. Our approach is defining an ideal functionality $\mathcal{F}_{\mathsf{FLPSI}}$ that outputs whatever Π_{FLPSI} protocol formally outputs. In particular, we formalize in functionality: giving a trusted party to run Π_{FLPSI} and giving to both parties what Π_{FLPSI} outputs. A similar issue regarding ideal functionalities is addressed in [CFR23].

Definition 3 (Security of a FLPSI protocol [CFR23]). *We say that a protocol $\Pi^{\delta,\mathcal{L}}_{\mathsf{FLPSI}}$ defined w.r.t. the closeness domain $\delta = (\mathcal{D}, \mathsf{CI})$ having the true positive rate TPR and the true negative rate TNR is a secure FLPSI protocol with leakage profile $\mathcal{L} = \{\mathcal{L}_{\mathsf{S}}, \mathcal{L}_{\mathsf{R}}\}$, if Π_{FLPSI} securely realizes the functionality $\mathcal{F}_{\mathsf{FLPSI}}$.*

Fig. 2: $\mathcal{F}_{\mathsf{FLPSI}}$ in semi-honest setting.

PARAMETERS:

- There are two parties, a receiver has input $x \in \mathcal{D}$ while a sender has a set of inputs $Y = \{(y_i, \ell_i)\}_{i\in[m]}$ where $y_i \in \mathcal{D}$ and $\ell_i \in \mathcal{LS}$.
- The protocol Π_{FLPSI} with closeness domain $\delta = (\mathcal{D}, \mathsf{CI})$ having the true positive rate TPR and the true negative rate TNR.
- The leakage profile $\mathcal{L} = \{\mathcal{L}_{\mathsf{S}}, \mathcal{L}_{\mathsf{R}}\}$.

FUNCTIONALITY:

- The trusted party runs an honest execution of the protocol Π_{FLPSI} on the players' inputs and obtains the output result set R;
- Return $(\mathsf{OutputR}, R, \mathcal{L}_{\mathsf{R}})$ to receiver and $(\mathsf{OutputS}, \mathcal{L}_{\mathsf{S}})$ to sender.

Given a leakage profile $\mathcal{L}$ that is allowed beforehand depending on the use context, the above security notion guarantees that no additional information beyond the output of the protocol Π and the leakage profile $\mathcal{L}$ is revealed. As discussed in [UCK+21], this security notion, which is a combination between simulation-based security and game-based security, helps to avoid the need to specify exactly the probability that CI outputs close or far for each pair of inputs in the ideal functionality. A protocol Π_{FLPSI} is said to be correct and secure if it satisfies both the above correctness and security notions.

Ideal functionality of TLPSI. A key building block to construct FLPSI is the threshold-labeled PSI (TLPSI). We give the ideal functionality of TLPSI against semi-honest adversaries in Figure 3.

Fig. 3: $\mathcal{F}^{d,t}_{\mathsf{TLPSI}}$ in semi-honest setting.

PARAMETERS:

- An arbitrary field $\mathbb{F}$, threshold value t.
- There are two parties, a receiver with a input set $X \subseteq \mathbb{F}$ and a sender with a input set $Y \subseteq \mathbb{F}$ with a secret label $l \in \mathbb{F}$.

FUNCTIONALITY:

- Wait for receiver to send the input set (InputR, X).
- Wait for sender to send the input set (InputS, Y, l).
- If $|X \cap Y| \geq t$, gives receiver $(\mathsf{OutputR}, X \cap Y, \ell)$, otherwise outputs $\perp$.

Masked Oblivious Pseudorandom Function. From our threshold-labeled PSI, we use a masked oblivious pseudorandom function (masked OPRF) to get an efficient fuzzy-labeled PSI.

The masked OPRF is a two-party protocol that is similar to OPRF except the receiver's input is masked by some masks given by the sender before it is used as the input to the PRF. Specifically, we consider a batch of T different masks applied to one input. The exact functionality is given in Figure 4. The same functionality appears in [UCK+21] and the authors refer to it as "oblivious sub-sampling". This functionality is straightforward to instantiate using a garbled circuit which we describe in Section 6.2.

Fig. 4: $\mathcal{F}^{T}_{\mathsf{mOPRF}}$ in the semi-honest setting.

PARAMETERS:

- Given a PRF function $F : \mathcal{K} \times \mathcal{M} \to \mathcal{D}$, a number T of instances, two parties a sender and a receiver.
- The receiver has an input $x \in \mathcal{M}$, the sender has a key $k \in \mathcal{K}$ and T masking values $\{m_1, m_2, \ldots, m_T\} \in \mathcal{M}^T$.

FUNCTIONALITY:

- Wait for receiver and sender to input their inputs (InputR, x) and $(\mathsf{InputS}, k, (m_i)_{i\in[T]})$ respectively.
- Output $(\mathsf{OutputR}, (F(k, x \wedge m_i))_{i\in[T]})$ to receiver.

3.5 Secure Two-Party Computation

In our work, all proofs are given in simulation-based proof and we give the formal definition of security under the semi-honest setting below.

Semi-honest security. Given a security parameter κ and a pair of inputs $(X, Y) \in \{0,1\}^*$, let $\mathrm{view}_1^{\Pi}(X, Y, \kappa)$ and $\mathrm{view}_2^{\Pi}(X, Y, \kappa)$ be the views of P_1 and P_2 respectively in the protocol Π, $\mathsf{out}^{\Pi}(X, Y, \kappa)$ be the output of the protocol, and $f_i(X, Y)$ be the output of P_i from the ideal functionality $f := (f_1, f_2)$. The protocol Π securely computes f in the presence of static semi-honest adversaries if there exist PPT simulators Sim_1 and Sim_2 such that for all valid inputs X, Y of f,

$$\big(\mathrm{view}_1^{\Pi}(X, Y, \kappa), \mathsf{out}^{\Pi}(X, Y, \kappa)\big) \approx (\mathsf{Sim}_1(1^{\kappa}, X, f_1(X, Y)), f(X, Y))\,;$$
$$\big(\mathrm{view}_2^{\Pi}(X, Y, \kappa), \mathsf{out}^{\Pi}(X, Y, \kappa)\big) \approx (\mathsf{Sim}_2(1^{\kappa}, Y, f_2(X, Y)), f(X, Y))\,.$$

3.6 Vector-OLE from PCG

We represent on Figure 5 the ideal functionality $\mathcal{F}^{n,\mathbb{F}}_{\mathsf{VOLE}}$ of vector-OLE[3]. In our concrete instantiations, we instantiate this functionality using the general template in [BCG+19b, BCG+19a, WYKW21], which can be instantiated under various flavors of the learning parity with noise (LPN) assumption. Using the protocol of [BCG+19a] to distribute the seeds[4], the theoretical communication cost to have $\mathcal{F}^{n,\mathbb{F}}_{\mathsf{VOLE}}$ is $(\log(cn/t)|\mathsf{OT}| + 10|\mathbb{F}|) \cdot t$, where t is Hamming weight of noises for the underlying LPN assumption, $|\mathsf{OT}| = O(\kappa)$ is the size of chosen OT [LWYY22] and c is the ratio between number of samples and the length of VOLE. We also provide an analysis of the cost of generating other correlations from PCG, specifically, ring-OLE and OLE in [BCG+20], to demonstrate the concrete efficiency of VOLE.

3.7 Threshold Shamir's Secret Sharing

Shamir's secret sharing allows each of d parties to get a share of the secret input such that this secret can only be revealed by at least t honest parties. The functionality is given in Figure 6.

[3] Since the role of sender and receiver are equal, for simplicity, we assume receiver holds global Δ.

[4] This protocol uses a length-t reverse VOLE protocol as a black box, which we instantiate with the construction of [ADI+17].

Fig. 5: $\mathcal{F}_{\mathsf{VOLE}}^{n,\mathbb{F}}$ ideal functionality.

PARAMETERS: Two parties, a sender and a receiver, an integer n, the size of the output vector, a finite field $\mathbb{F}$.
FUNCTIONALITY:

- **Initialize:** Upon receiving $(\mathsf{init}, \mathsf{InputS})$ and $(\mathsf{init}, \mathsf{InputR})$ from sender and receiver, sample $\Delta \leftarrow\$ \mathbb{F}$ if receiver is honest or receive $\Delta \in \mathbb{F}$ from the adversary otherwise. It stores global key Δ and send Δ to receiver, and ignores all subsequent init commands.
- **Extend:** This procedure can be run multiple times. Upon receiving $(\mathsf{extend}, \mathsf{n})$ from sender and receiver, do:
  1. If receiver is corrupted then wait for $\mathcal{A}$ to send $\mathbf{w} \in \mathbb{F}^n$; samples $\mathbf{u} \leftarrow\$ \mathbb{F}^n$ and computes $\mathbf{v} := \mathbf{w} - \Delta \cdot \mathbf{u}$.
  2. If sender is corrupted then wait for $\mathcal{A}$ to send vectors $\mathbf{u} \in \mathbb{F}^n, \mathbf{v} \in \mathbb{F}^n$ and computes $\mathbf{w} := \Delta \cdot \mathbf{u} + \mathbf{v}$.
  3. Otherwise, samples $\mathbf{u} \leftarrow\$ \mathbb{F}^n, \mathbf{v} \leftarrow\$ \mathbb{F}^n$ and computes $\mathbf{w} := \Delta \cdot \mathbf{u} + \mathbf{v}$.
- **Return:** Functionality sends $(\mathsf{OutputS}, \mathbf{u}, \mathbf{v})$ to sender and $(\mathsf{OutputR}, \Delta, \mathbf{w})$ to receiver.

Fig. 6: $\mathcal{F}_{\mathsf{TSSS}}^{d,t}$ over $\mathbb{F}$.

PARAMETERS: Given a field $\mathbb{F}$, a threshold value t, and d distinct points $t_i \in \mathbb{F}$ such that $t_i \neq 0$.

FUNCTIONALITY:

- **Share.** For a secret input $s \in \mathbb{F}$, pick a random polynomial f of degree $(t-1)$ over $\mathbb{F}[x]$ such that $f(0) = s$. Output the set $S := \{f(t_i)\}_{i\in[d]}$.
- **Reconstruct.** Given a vector $\mathbf{c} := \{c_i\}_{i\in[d]}$, if there are at least t value c_i such that $c_i = f(t_i)$ for $i \in [d]$, reconstruct f and output $s := f(0)$. Otherwise, output $\perp$.

4 Vector Ring-OLE and its Applications

We propose a new primitive called vector ring-OLE that allows receiver and sender to execute the batch of chosen polynomial OLE where receiver's input in each instance is the same, its ideal functionality $\mathcal{F}_{\mathsf{vrOLE}}^{m,d}$ and its construction $\Pi_{\mathsf{vrOLE}}^{m,d}$ are described in Figure 7, Figure 8, respectively. Our construction offers *two advantages*: (1) the de-randomization communication cost is cheaper compared to prior works, (2) faster setup phase compared to ring-OLE protocols. We leverage the efficiency of our construction to develop our main contribution—fuzzy labeled PSI.

4.1 Ideal Functionality of Vector Ring-OLE

Compared with the standard ring-OLE [BCG+20], the sender and the receiver in our construction can choose components of the OLE relation ahead of time and the operation is computed without reduction, i.e., we do not work in a quotient ring. Particularly, receiver will choose an input $u_0 \in \mathbb{F}[x]$ of degree d and sender will choose an input $\{v_i\}_{i\in[m]} \in \mathbb{F}[x]$ of degree $2d$, then receiver obtains $\{z_i\}_{i\in[m]} \in \mathbb{F}[x]$ such that $z_i = u_0 \cdot u_i + v_i \in \mathbb{F}[x]$ (degree $2d$) while sender gets $\{u_i\}_{i\in[m]} \in \mathbb{F}[x]$ that are set of random degree-d polynomials. The ideal functionality $\mathcal{F}_{\mathsf{vrOLE}}^{m,d}$ is shown in Figure 7.

4.2 Vector Ring-OLE

We assume that before executing the protocol, there is a trusted third-party[5] samples randomly $2d+1$ distinct evaluation points $(x_j)_{j\in[2d+1]} \leftarrow\$ \mathbb{F}^{2d+1}$. The concept of the setup phase is widely recognized and utilized in many other related PSI works [ADT11, GN19, ABD+21, ALOS22]. We then instantiate our semi-honest $\mathcal{F}_{\mathsf{vrOLE}}^{m,d}$ using $\mathcal{F}_{\mathsf{VOLE}}^{m,\mathbb{F}}$, resulting in a total communication cost of just $m(3d+1)\log|\mathbb{F}|$. The technical overview for distributing correlation is shown in the Section 2.1. We show how to build a simulator to interact with semi-honest adversaries by a hybrid proof. The detailed protocol and theorem for correctness and security are given in Figure 8 and Theorem 4.

[5] The setup can be reused multiple times by any number of senders and receivers.

Fig. 7: $\mathcal{F}_{\mathsf{vrOLE}}^{m,d}$ in semi-honest setting.

PARAMETERS: Given a finite field $\mathbb{F}$, a polynomial ring $\mathcal{R} = \mathbb{F}[x]$, length d and instance count m, a set of $(2d+1)$ random elements $(x_j)_{j\in[2d+1]}$ over $\mathbb{F}$; the protocol is executed between two parties, a sender and a receiver.

- Receiver has an input polynomial $u_0 \neq 0$ of degree less than $d+1$ over $\mathcal{R}$.
- Sender has an input of m polynomials $(v_i)_{\in[m]} \neq 0$ of degree less than $2d+1$ over $\mathcal{R}$.

FUNCTIONALITY:

1. Waiting for inputs (InputR, u_0) and $(\mathsf{InputS}, (v_i)_{i\in[m]})$ from receiver and sender, respectively:
   - If receiver is corrupted then verify that $u_0 \in \mathcal{R},\ u_0 \neq 0$. else abort the protocol.
   - If sender is honest, sample $(u_i)_{i\in[m]} \leftarrow\$\ \mathcal{R}$ of degree d.
     Otherwise, sender is corrupted, wait for sender to send $(\mathsf{InputS}, (u_i)_{i\in[m]})$.
2. Compute z_i is a polynomial of degree $2d$ such that $z_i(x_j) := u_0(x_j)\cdot u_i(x_j) + v_i(x_j)$ for $i \in [m],\ j \in [2d+1]$.
3. Output $(\mathsf{OutputS}, (u_i)_{i\in[m]})$ to sender and $(\mathsf{OutputR}, (z_i)_{i\in[m]})$ to receiver.

Theorem 4. *The protocol $\Pi_{\mathsf{vrOLE}}^{m,d}$ in Figure 8 securely realizes the ideal functionality $\mathcal{F}_{\mathsf{vrOLE}}^{m,d}$ for the semi-honest setting and in the $\mathcal{F}_{\mathsf{VOLE}}^{m,\mathbb{F}}$-hybrid model.*

Fig. 8: $\Pi_{\mathsf{vrOLE}}^{m,d}$ from $\mathcal{F}_{\mathsf{VOLE}}^{m,\mathbb{F}}$ in semi-honest setting.

PARAMETERS AND INPUTS:

- Defines polynomial ring $\mathcal{R} = \mathbb{F}[X]$, and $2d+1$ distinct evaluation points $(x_i)_{i\in[2d+1]} \in \mathbb{F}^{2d+1}$.
- Receiver has a polynomial input $u_0 \in \mathcal{R}$ of degree d, sender has a set input of m polynomials $(v_i)_{i\in[m]} \in \mathcal{R}^m$ of degree $2d$.
- An instantiation of vector-OLE $\mathcal{F}_{\mathsf{VOLE}}^{m,\mathbb{F}}$.

PROTOCOL:

- **Preprocessing phase:**
  1. Sender with input $(\mathsf{init}, \mathsf{InputS})$ and receiver with input $(\mathsf{init}, \mathsf{InputR})$ invoke $\mathcal{F}_{\mathsf{VOLE}}^{m,\mathbb{F}}$ $2d+1$ times, for each time $i \in [1, 2d+1]$:
     - Sender gets $\mathbf{U}'_i \in \mathbb{F}^m,\ \mathbf{V}'_i \in \mathbb{F}^m$.
     - Receiver gets $u'_{0,i} \in \mathbb{F},\ \mathbf{Z}'_i := u'_{0,i}\cdot\mathbf{U}'_i + \mathbf{V}'_i \in \mathbb{F}^m$.
  2. Receiver defines $\mathbf{u}'_0 := \{u'_{0,1}, \ldots, u'_{0,2d+1}\} \in \mathbb{F}^{2d+1}$, $\mathbf{Z}' \in \mathbb{F}^{m\times(2d+1)}$, sender defines $\mathbf{U}' \in \mathbb{F}^{m\times(2d+1)},\ \mathbf{V}' \in \mathbb{F}^{m\times(2d+1)}$ such that $\mathbf{U}'_i, \mathbf{V}'_i$ and $\mathbf{Z}'_i$ are respectively columns of $\mathbf{U}', \mathbf{V}'$ and $\mathbf{Z}'$. Note that $\mathbf{Z}' := \mathbf{u}'_0 \odot \mathbf{U}' + \mathbf{V}'$ where $\odot$ is the point-wise product.
  3. From matrix $\mathbf{U}'[1, d+1] \in \mathbb{F}^{m\times(d+1)}$, sender defines the set of m degree-d polynomials $(u_i)_{i\in[m]} \in \mathcal{R}^m$ as below:
     - For each row $i \in [1, m]$ of $\mathbf{U}'[1, d+1]$, defines a degree-d polynomial u_i such that $(u_i(x_1), \ldots, u_i(x_{d+1})) = \mathbf{U}'^i[1, d+1]$.
       // $(u_i)_{i\in[m]}$ is unique and is defined by interpolation of first $(d+1)$ columns of $\mathbf{U}'$.
  4. Sender defines two new matrices $\Delta\mathbf{U}$, $\mathbf{U}$:
     - Defines a matrix $\mathbf{U} \in \mathbb{F}^{m\times d}$, such that the column $\mathbf{U}_i := \{u_1(x_{i+d+1}), \ldots, u_m(x_{i+d+1})\} \in \mathbb{F}^m$ for $i \in [1, d]$.
     - Compute a matrix $\Delta\mathbf{U} := \mathbf{U} - \mathbf{U}'[d+2, 2d+1] \in \mathbb{F}^{m\times d}$.
- **Online phase:**
  1. Sender defines a matrix $\mathbf{V} \in \mathbb{F}^{m\times(2d+1)}$ such that its columns are defined as $\mathbf{V}_i := \{v_1(x_i), \ldots, v_m(x_i)\} \in \mathbb{F}^m$ for $i \in [1, 2d+1]$.
  2. Receiver computes $2d+1$ evaluations $\mathbf{u}_0 := \{u_0(x_1), \ldots, u_0(x_{2d+1})\} \in \mathbb{F}^{2d+1}$, and defines $\Delta\mathbf{u}_0 := \mathbf{u}_0 - \mathbf{u}'_0 \in \mathbb{F}^{2d+1}$ then sends $\Delta\mathbf{u}_0$ to sender.
  3. Sender defines $\Delta\mathbf{V} := \Delta\mathbf{u}_0 \odot \mathbf{U}' + \mathbf{V} - \mathbf{V}' \in \mathbb{F}^{m\times(2d+1)}$ and sends $\Delta\mathbf{U}, \Delta\mathbf{V}$ to receiver.
     // $\Delta\mathbf{U}$ is defined in the preprocessing phase, $\mathbf{V} - \mathbf{V}'$ can be computed in preprocessing phase.

4. Receiver defines $\mathbf{Z} \in \mathbb{F}^{m\times(2d+1)}$ as follows:
   - $\mathbf{Z}[1, d+1] = \Delta\mathbf{V}[1, d+1] + \mathbf{Z}'[1, d+1] \in \mathbb{F}^{m\times(d+1)}$.
   - $\mathbf{Z}[d+2, 2d+1] := \mathbf{u}_0[d+1, 2d+1] \odot \Delta\mathbf{U} + \Delta\mathbf{V}[d+1, 2d+1] + \mathbf{Z}'[d+1, 2d+1] \in \mathbb{F}^{m\times d}$.

   Finally, receiver defines the polynomials $z_i(x)$ of degree $2d$ by polynomial interpolation of $2d+1$ points $\{(x_j, \mathbf{Z}_{i,j})\}_{j\in[1,2d+1]}$. Note here $z_i(x) = u_0(x) \cdot u_i(x) + v_i(x)$ for $i \in [1, m]$.

Proof (Proof of Theorem 4). Our proof consists of two parts, correctness and security.

Correctness. Firstly, we show that our protocol in Figure 8 realizes the correctness of the ideal functionality $\mathcal{F}^{m,d}_{\mathsf{vrOLE}}$. In particular, we obtain $\mathcal{F}^{m,d}_{\mathsf{vrOLE}}$ with the input of receiver u_0 of degree d is the same in m instances while the sender has a set input of m polynomials $(u_i)_{i\in[m]} \in \mathcal{R}^m$ of degree d and m polynomials $(v_i)_{i\in[m]} \in \mathcal{R}^m$ of degree $2d$; in the end of protocol receiver gets $(z_i := u_0 \cdot u_i + v_i)_{i\in[m]}$, where the degree of $(z_i)_{i\in[m]}$ is $2d$.

Indeed, observe that $(u_i)_{i\in[m]}$ is defined from the first $(d+1)$ columns of matrix $\mathbf{U}'$, therefore, we do not need to de-randomize these columns and can reuse them; the matrix $\mathbf{Z} \in \mathbb{F}^{m\times(2d+1)}$ is computed in two parts, first $(d+1)$ columns and last d columns, we can formalize as:

$$
\begin{aligned}
\mathbf{Z} &= 0^{d+1} \| (\mathbf{u}_0[d+2, 2d+1] \odot \Delta\mathbf{U}) + \Delta\mathbf{V} + \mathbf{Z}' \\
&= 0^{d+1} \| (\mathbf{u}_0[d+2, 2d+1] \odot (\mathbf{U} - \mathbf{U}'[d+2, 2d+1])) \\
&\quad + (\Delta\mathbf{u}_0 \odot \mathbf{U}' + \mathbf{V} - \mathbf{V}') + (\mathbf{u}'_0 \odot \mathbf{U}' + \mathbf{V}') \\
&= 0^{d+1} \| (\mathbf{u}_0[d+2, 2d+1] \odot (\mathbf{U} - \mathbf{U}'[d+2, 2d+1])) \\
&\quad + ((\mathbf{u}_0 - \mathbf{u}'_0) \odot \mathbf{U}' + \mathbf{V} - \mathbf{V}') + (\mathbf{u}'_0 \odot \mathbf{U}' + \mathbf{V}') \\
&= 0^{d+1} \| (\mathbf{u}_0[d+2, 2d+1] \odot (\mathbf{U} - \mathbf{U}'[d+2, 2d+1])) + \mathbf{u}_0 \odot \mathbf{U}' + \mathbf{V} \\
&= 0^{d+1} \| (\mathbf{u}_0[d+2, 2d+1] \odot \mathbf{U}) + (\mathbf{u}_0[1, d+1] \odot \mathbf{U}'[1, d+1] \| 0^d) + \mathbf{V} \\
&= \mathbf{u}_0 \odot (\mathbf{U}'[1, d+1] \| \mathbf{U}) + \mathbf{V}.
\end{aligned}
$$

We recall, $\mathbf{u}_0 := \{u_0(x_1), \ldots, u_0(x_{2d+1})\} \in \mathbb{F}^{2d+1}$, and if we define $\mathbf{M} = (\mathbf{U}'[1, d+1] \| \mathbf{U}) \in \mathbb{F}^{m\times(2d+1)}$ then matrices $\mathbf{M}, \mathbf{V}, \mathbf{Z} \in \mathbb{F}^{m\times(2d+1)}$ as the column $\mathbf{M}_j$ defined by $\{u_1(x_j), u_2(x_j), \ldots, u_m(x_j)\}$, and $\mathbf{V}_j, \mathbf{Z}_j$ are defined the same way as $\mathbf{M}_j$ from the polynomials $(v_i)_{i\in[m]}$ and $(z_i)_{i\in[m]}$, we then argue that $\mathbf{Z} = \mathbf{u}_0 \odot \mathbf{U} + \mathbf{V}$. It means for row $\mathbf{Z}^i$ of matrix $\mathbf{Z}$, we have $\mathbf{Z}^i = \mathbf{u}_0 \odot \mathbf{U}^i + \mathbf{V}^i \implies \mathbf{Z}^i = (u_0(x_1) \cdot u_i(x_1) + v_i(x_1), u_0(x_2) \cdot u_i(x_2) + v_i(x_2), \ldots, u_0(x_{2d+1}) \cdot u_i(x_{2d+1}) + v_i(x_{2d+1})) \in \mathbb{F}^{2d+1}$. So the polynomial z_i of degree $2d$ is well-defined by polynomial interpolation of $2d+1$ points $\{(x_j, \mathbf{Z}_{i,j})\}_{j\in[1,2d+1]}$.

Security. We prove our security in the simulation-based model by showing there exists a simulator Sim who can simulate the view of $\mathcal{A}$ in two cases.

Corrupted receiver.

- Sim emulates the $\mathcal{F}^{m,\mathbb{F}}_{\mathsf{VOLE}}$ and gets the input $\mathbf{u}'_0$ from $\mathcal{A}$. Sim samples randomly matrices $\mathbf{U}', \mathbf{V}' \leftarrow\$ \mathbb{F}^{m\times(2d+1)}$ then sends $\mathbf{Z}' := \mathbf{u}'_0 \odot \mathbf{U}' + \mathbf{V}' \in \mathbb{F}^{m\times(2d+1)}$ to $\mathcal{A}$.
- Sim acts as an honest sender and receives $\Delta\mathbf{u}_0 = \mathbf{u}_0 - \mathbf{u}'_0$, Sim defines $\mathbf{u}_0 := \Delta\mathbf{u}_0 + \mathbf{u}'_0$.
- Sim defines a polynomial u_0 as the interpolation of $2d+1$ points $(x_i, \mathbf{u}_{0,i})$.

- Sim acts as an honest sender in the real protocol.
  - Sim joins $\mathcal{F}^{m,d}_{\mathsf{vrOLE}}$ as a receiver with input u_0 then Sim gets a set of polynomials $\{z_i\}_{i\in[m]}$. Sim defines a matrix $\mathbf{Z}$ such that each row

$$\mathbf{Z}^i := (z_i(x_1), z_i(x_2), \ldots, z_i(x_{2d+1})) \in \mathbb{F}^{2d+1}.$$

  - Sim defines set of degree-d polynomials $(u_i)_{i\in[m]}$ over $\mathcal{R}^m$ from $\mathbf{U}'[1, d+1]$ by interpolation using $(d+1)$ points $(x_1, \ldots, x_{d+1})$ and then defines a matrix $\mathbf{U}$ such that the column

$$\mathbf{U}_i := \{u_1(x_{d+1+i}), u_2(x_{d+1+i}), \ldots, u_m(x_{d+1+i})\} \in \mathbb{F}^m$$

    for $i \in [1, d]$.
    then sends a matrix $\Delta\mathbf{U} := \mathbf{U} - \mathbf{U}'[d+2, 2d+1] \in \mathbb{F}^{m\times d}$ to $\mathcal{A}$.

* Sim computes $\Delta\mathbf{V} := \mathbf{Z} - \mathbf{u}_0 \odot (0^{d+1} \| \Delta\mathbf{U}) - \mathbf{Z}' \in \mathbb{F}^{m\times(2d+1)}$. On behalf of sender, Sim sends $\Delta\mathbf{V}$ to $\mathcal{A}$.

The simulation against corrupted receiver is indistinguishable from the real protocol as following hybrids:

- Hyb 0. The same as the real protocol with an honest sender and $\mathbb{F}_{\mathsf{VOLE}}^{m,\mathbb{F}}$ is executed honestly.
- Hyb 1. Sim emulates the $\mathcal{F}_{\mathsf{VOLE}}^{m,\mathbb{F}}$ and gets the input $\mathbf{u}_0'$ from $\mathcal{A}$. Sim samples randomly matrices $\mathbf{U}', \mathbf{V}' \leftarrow\$\ \mathbb{F}^{m\times(2d+1)}$ then sends $\mathbf{Z}' := \mathbf{u}_0' \odot \mathbf{U}' + \mathbf{V}' \in \mathbb{F}^{m\times(2d+1)}$ to $\mathcal{A}$. Sim simulates the sender and receives $\Delta\mathbf{u}_0 = \mathbf{u}_0 - \mathbf{u}_0'$, Sim defines $\mathbf{u}_0 := \Delta\mathbf{u}_0 + \mathbf{u}_0'$. Sim defines a polynomial u_0 as the interpolation of $2d+1$ points $(x_i, \mathbf{u}_{0,i})$.
  This hybrid is indistinguishable from the previous hybrid since the output of $\mathcal{F}_{\mathsf{VOLE}}^{m,\mathbb{F}}$ are pseudo-random in the view of $\mathcal{A}$.
- Hyb 3. Sim simulate as a sender in real protocol.
  * Sim joins $\mathcal{F}_{\mathsf{vrOLE}}^{m,d}$ as a receiver with input u_0 then Sim gets a set of polynomials $\{z_i\}_{i\in[m]}$. Sim defines a matrix $\mathbf{Z}$ such that each row

$$\mathbf{Z}^i := (z_i(x_1), z_i(x_2), \ldots, z_i(x_{2d+1})) \in \mathbb{F}^{2d+1}.$$

  * Sim defines set of degree-d polynomials $(u_i)_{i\in[m]}$ over $\mathcal{R}^m$ from $\mathbf{U}'[1, d+1]$ by interpolation using $(d+1)$ points $(x_1, \ldots, x_{d+1})$ and then defines a matrix $\mathbf{U}$ such that the column

$$\mathbf{U}_i := \{u_1(x_{d+1+i}), u_2(x_{d+1+i}), \ldots, u_m(x_{d+1+i})\} \in \mathbb{F}^m$$

    for $i \in [1, d]$.
    then sends a matrix $\Delta\mathbf{U} := \mathbf{U} - \mathbf{U}'[d+2, 2d+1] \in \mathbb{F}^{m\times d}$ to $\mathcal{A}$.
    This hybrid is indistinguishable from the previous hybrid since in real execution $\Delta\mathbf{U}$ is randomized by $\mathbf{U}'$ and $\mathbf{U}' \leftarrow\$\ \mathbb{F}^{m\times(2d+1)}$ so in the view of $\mathcal{A}$, $\Delta\mathbf{U}$ is random vector.
- Hyb 4. Sim computes $\Delta\mathbf{V} := \mathbf{Z} - \mathbf{u}_0 \odot (0^{d+1} \| \Delta\mathbf{U}) - \mathbf{Z}' \in \mathbb{F}^{m\times(2d+1)}$. On behalf of sender, Sim sends $\Delta\mathbf{V}$ to $\mathcal{A}$.
  This hybrid is indistinguishable from the previous hybrid. Indeed, the output of $\mathcal{A}$ is identical to the output of ideal functionality by the correctness of protocol. Moreover, $\Delta\mathbf{V}$ is randomized by $\mathbf{V}'$ which is sampled uniformly random so $\Delta\mathbf{V}$ looks random in the view of $\mathcal{A}$.

Corrupted sender.

- Sim emulates the $\mathcal{F}_{\mathsf{VOLE}}^{m,\mathbb{F}}$ and gets the input $\mathbf{U}', \mathbf{V}'$ from $\mathcal{A}$, samples $\mathbf{u}_0' \leftarrow\$\ \mathbb{F}^{2d+1}$ and defines $\mathbf{Z}' := \mathbf{u}_0' \odot \mathbf{U}' + \mathbf{V}'$.
- Sim samples randomly a polynomial u_0 being different zero of degree d and defines $\Delta\mathbf{u}_0 = \mathbf{u}_0 + \mathbf{u}_0'$, then sends it to $\mathcal{A}$.
- Sim plays the role of receiver, and learns $\Delta\mathbf{U}, \Delta\mathbf{V}$ from $\mathcal{A}$.
  From $\Delta\mathbf{U}$, Sim defines $\mathbf{U} := \Delta\mathbf{U} + \mathbf{U}'[d+2, 2d+1] \in \mathbb{F}^{m\times d}$.
  From $\Delta\mathbf{V}$, Sim defines $\mathbf{V} := \Delta\mathbf{V} + \mathbf{V}' - \Delta\mathbf{u}_0 \odot \mathbf{U}' \in \mathbb{F}^{m\times(2d+1)}$.
- Define a matrix $\mathbf{M} := (\mathbf{U}'[1, d+1] \| \mathbf{U})$, for each row $\mathbf{M}^i, \mathbf{V}^i$ of $\mathbf{M}, \mathbf{V}$, Sim defines a polynomial u_i, v_i as the interpolation polynomial of a set of points $\{(x_j, \mathbf{M}^i[j])\}_{j\in[2d+1]}$ and $\{(x_j, \mathbf{V}^i[j])\}_{j\in[2d+1]}$ respectively.
- Sim joins $\mathcal{F}_{\mathsf{vrOLE}}^{m,d}$ as a sender with input $\{u_i, v_i\}_{i\in[m]}$ and outputs whatever $\mathcal{F}_{\mathsf{vrOLE}}^{m,d}$ outputs.

The simulation against corrupted sender is indistinguishable from the real protocol as following hybrids:

- Hyb 0. The same as the real protocol with an honest sender and $\mathbb{F}_{\mathsf{VOLE}}^{m,\mathbb{F}}$ is executed honestly.
- Hyb 1.Sim emulates the $\mathcal{F}_{\mathsf{VOLE}}^{m,\mathbb{F}}$ and gets the input $\mathbf{U}', \mathbf{V}'$ from $\mathcal{A}$, samples $\mathbf{u}_0' \leftarrow\$\ \mathbb{F}^{2d+1}$ and defines $\mathbf{Z}' := \mathbf{u}_0' \odot \mathbf{U}' + \mathbf{V}'$.
  This hybrid is indistinguishable from the previous hybrid since the output of $\mathcal{F}_{\mathsf{VOLE}}^{m,\mathbb{F}}$ are pseudorandom in the view of $\mathcal{A}$.
- Hyb 3. Sim samples randomly a degree-d polynomial u_0 being different zero and defines $\Delta\mathbf{u}_0 = \mathbf{u}_0 - \mathbf{u}_0'$, then sends it to $\mathcal{A}$. This hybrid is indistinguishable from the previous hybrid since in real execution $\mathbf{u}_0$ is randomized by $\mathbf{u}_0'$ and $\mathbf{u}_0' \leftarrow\$\ \mathbb{F}^{2d+1}$ so in the view of $\mathcal{A}$, $\Delta\mathbf{u}_0$ is random vector.

- Hyb 4. Sim plays the role of sender, and learns $\Delta\mathbf{U}, \Delta\mathbf{V}$ from $\mathcal{A}$.
  From $\Delta\mathbf{U}$, Sim defines $\mathbf{U} := \Delta\mathbf{U} + \mathbf{U}'[d+2, 2d+1] \in \mathbb{F}^{m\times d}$.
  From $\Delta\mathbf{V}$, Sim defines $\mathbf{V} := \Delta\mathbf{V} + \mathbf{V}' - \Delta\mathbf{u}_0 \odot \mathbf{U}' \in \mathbb{F}^{m\times(2d+1)}$.
  Define a matrix $\mathbf{M} := (\mathbf{U}'[1, d+1] \| \mathbf{U})$, for each row $\mathbf{M}^i, \mathbf{V}^i$ of $\mathbf{M}, \mathbf{V}$, Sim defines a polynomial u_i, v_i as the interpolation polynomial of a set of points $\{(x_j, \mathbf{M}^i_j)\}_{j\in[2d+1]}$ and $\{(x_j, \mathbf{V}^i_i)\}_{j\in[2d+1]}$ respectively. Sim joins $\mathcal{F}^{m,d}_{\mathsf{vrOLE}}$ as a sender with input $\{u_i, v_i\}_{i\in[m]}$ and outputs whatever $\mathcal{F}^{m,d}_{\mathsf{vrOLE}}$ outputs.
  This hybrid is indistinguishable from the previous hybrid. □

4.3 Efficiency of Vector Ring-OLE

Communication. The communication complexity of our vector ring-OLE consists of the offline cost for setting up $\mathcal{F}^{m,\mathbb{F}}_{\mathsf{VOLE}}$ $(2d+1)$ times and the online cost which is primarily the de-randomizing step (sending $\Delta\mathbf{u}_0, \Delta\mathbf{U}, \Delta\mathbf{V}$). In our setting, m is significantly larger than d so the dominant communication cost is $m(3d+1)\log|\mathbb{F}|$ bits, the exact total number of online bits communicated is

$$\underbrace{(2d+1)\log|\mathbb{F}|}_{\Delta\mathbf{u}_0} + \underbrace{md\log|\mathbb{F}|}_{\Delta\mathbf{U}} + \underbrace{m(2d+1)\log|\mathbb{F}|}_{\Delta\mathbf{V}}$$

Computation. In the preprocessing phase, we consider the computation cost of the sender and the receiver (when invoking $2d+1$ instances of $\mathcal{F}^{m,\mathbb{F}}_{\mathsf{VOLE}}$) is minor since the instantiation of $\mathcal{F}^{m,\mathbb{F}}_{\mathsf{VOLE}}$ is efficient [WYKW21]. While in malicious setting, we need $4m\mathcal{F}_{\mathsf{OLE}}$ instances setup which is more expensive compared to VOLE setup. However, this cost is independent on d and if we use instantiation of $\mathcal{F}_{\mathsf{rOLE}}$ in [BCG+20] the computation cost for this setup is small. Therefore, the dominant computation cost in this phase comes from the polynomial interpolation to obtain m polynomials of degree d over $\mathbb{F}$.

In the online phase, the sender's computation is dominated by polynomial evaluation of m polynomials on $(2d+1)$ points over $\mathbb{F}$ and linear operations on vector and matrix over $\mathbb{F}^{m\times(2d+1)}$. The receiver's computation is dominated by interpolating m polynomials of degree $2d$ on $(2d+1)$ points. As long as our vector ring-OLE is not used in applications that require a large d, this is a reasonable cost as shown in our fuzzy-labeled PSI construction Section 5.

Comparison. In our setting, we require only $(2d+1)$ $\mathcal{F}^{m,\mathbb{F}}_{\mathsf{VOLE}}$ set up, where m is large (possibly millions) and d is small (e.g., 64). We compare with related work [GN19, CILO22], and the results are given in Table 1. Specifically, we use the same mechanism as in [CILO22] to build a batch of m chosen ring-OLE of degree-d and then compute its theoretical performance in our setting. To instantiate this construction in our setting, it requires m PCG-based ring-OLE of degree-$(2d+1)$ which can be instantiated from $2md$ $\mathcal{F}_{\mathsf{OLE}}$ by using interpolation polynomial, where $\mathcal{F}_{\mathsf{OLE}}$ can be obtained from $\mathcal{F}_{\mathsf{rOLE}}$.

Table 1. The communication of online and offline phase for several chosen ring-OLE protocol over field $\mathbb{F}$ [GN19, CILO22]. All protocols have m instantiations, and we omit minor communication costs that are independent of m. We denote d as the degree of polynomial in ring-OLE correlation.

Protocol	Online	Offline	Assumption
Our vrOLE	$m(3d+1)\log\|\mathbb{F}\|$	$(2d+1)\mathcal{F}^{m,\mathbb{F}}_{\mathsf{VOLE}}$	LPN
GN19 [GN19]	$9md\log\|\mathbb{F}\|$	$4md\mathcal{F}^{\mathbb{F}}_{\mathsf{OLE}}$	FHE
CILO22 [CILO22]	$m(4d+3)\log\|\mathbb{F}\|$	$2md\mathcal{F}^{\mathbb{F}}_{\mathsf{OLE}}$ $+2md\log\|\mathbb{F}\|$	Ring-LPN

5 Unbalanced Fuzzy-Labeled PSI

The primary building block of our fuzzy-labeled PSI is a threshold-labeled PSI which we describe in Section 5.1. Specifically, receiver and sender each have an input set, and sender additionally has a label ℓ. At the end of the protocol execution receiver receives ℓ if the intersection of their sets is greater than some threshold t. Our construction is efficient when the threshold parameter t is small ($t = O(1)$) and we show in Section 5.2 that it is sufficient for us to build an effective model of fuzzy PSI using masked OPRF.

5.1 Threshold-labeled PSI from Vector Ring-OLE

Instantiation. In this section, we present a new, efficient threshold-labeled PSI which is built from our vector ring-OLE and a threshold secret sharing scheme. Receiver has an input set X and sender has an input set Y where $0 \notin Y$ and a secret label l where $|X| = |Y| = d$. At the end of the protocol, receiver learns $X \cap Y$ and l if $|X \cap Y| \geq t$ otherwise receiver learns nothing. The main idea of our protocol is that receiver and sender join the $\mathcal{F}_{\mathsf{vrOLE}}^{1,d}$ with input u_0 and (u_1, v_1), respectively, where $u_0(x) = \prod_{i=1}^{d}(x - x_i)$, u_1 is uniformly random of degree d and v_1 is a polynomial of degree $2d$ such that $v_1(y) = f(y)$ for all $y \in Y$ where f is polynomial degree of t defined by the secret label $\ell'_B = \mathsf{PRF}_k(\ell)$ from $\mathcal{F}_{\mathsf{TSSS}}^{d,t}$ ($f(0) = \ell'_B$). By the definition of v_1, we have $v_1(x) = f(x) + \prod_{i=1}^{d}(x - y_i) \cdot R(x)$ where R is uniformly random polynomial of degree d. At the end of protocol, receiver gets a polynomial $v_0 := u_0 \cdot u_1 + v_1 = u_0 \cdot u_1 + \prod_{i=1}^{d}(x - y_i) \cdot R(x) + f(x)$ if the receiver's polynomial input being different to 0 has the degree less than d, otherwise receiver learns nothing.

To reconstruct ℓ, receiver sends $h_\ell = \mathcal{H}_1(\ell'_B)$ and $\Delta\ell = \mathcal{H}_2(\ell'_B) + \ell$ to sender. Receiver computes $V_0 := \{v_0(x_i)\}_{i\in[d]}$ and tries each combination of t elements in V_0 to recover ℓ'_A from $\mathcal{F}_{\mathsf{TSSS}}^{d,t}$. Receiver checks if $h_\ell = \mathcal{H}_1(\ell'_A)$, then recomputes $\ell = \Delta\ell - \mathcal{H}_2(\ell'_A)$ and ℓ is the correct label of sender. Since ℓ'_B is random in the view of sender (output of a PRF), the receiver learns no information about ℓ from h_ℓ and $\Delta\ell$.

Notably, it is not enough if the sender sends a digest of the label $h_\ell := \mathcal{H}(\ell)$ and then the receiver determines whether it obtained the correct label but check $h_\ell = \mathcal{H}(\ell')$, where ℓ' is the output of the reconstruction. The label domain may be small and a corrupted receiver is able to brute force the pre-image. Our construction does not have the aforementioned issue.

An alternative TLPSI instantiation that avoids expensive trial-reconstruction for a concrete parameter $t = 2$ (parameter we used for fuzzy construction) can be found in Appendix A.

To guarantee the security we need to show that receiver learns nothing about the sender's input Y and the label l is kept secret. Since $f(0) = \ell'_B = \mathsf{PRF}_k(\ell)$ (k is kept secret by receiver) and if receiver learns ℓ'_B then it can reconstruct ℓ, we need to make sure v_0 reveals nothing about $f(0)$. We assume $0 \notin Y$ which means $\prod_{i=1}^{d}(0 - y_i) \cdot R(0)$ since the constant term in R is a random element. Note that $v_0(0) = u_0(0) \cdot u_1(0) + \ell'_B + \prod_{i=1}^{d}(0 - y_i) \cdot R(0)$ so ℓ'_B is hidden by the random element in $\mathbb{F}$. Secondly, since receiver's input to $\Pi_{\mathsf{TLPSI}}^{d,t}$ is the same as in $\mathcal{F}_{\mathsf{vrOLE}}^{1,d}$, Sim can extract actual input of $\mathcal{A}$ (corrupted receiver) and then interact with $\mathcal{A}$ to simulate the views of $\mathcal{A}$. We show our protocol in Figure 9, the formal proof of correctness and security in Theorem 5, and the details of proof can be found in Theorem 5.

Fig. 9: $\Pi_{\mathsf{TLPSI}}^{d,t}$ from $\mathcal{F}_{\mathsf{vrOLE}}^{m,d}$ and $\mathcal{F}_{\mathsf{TSSS}}^{d,t}$.

PARAMETERS:

- Receiver has input set $X \in \mathbb{F}^d$, sender has input set $Y \in \mathbb{F}^d$ and a secret label $\ell \in \mathbb{F}$ and $0 \notin Y$. The cardinality of two sets is $|X| = |Y| = d$. The threshold value is $t \leq d$.
- A vector ring-OLE $\mathcal{F}_{\mathsf{vrOLE}}^{1,d}$ followed ideal functionality of $\mathcal{F}_{\mathsf{vrOLE}}$ over $\mathbb{F}[x]$ (Figure 7).
- A threshold Shamir's secret sharing $\mathcal{F}_{\mathsf{TSSS}}^{d,t}$ over $\mathbb{F}$.
- Collision-resistant hash functions $\mathcal{H}_1, \mathcal{H}_2 : \{0,1\}^* \to \{0,1\}^\kappa$.

PROTOCOL:

- **Preprocessing phase:**
  1. For $X = \{x_i\}_{i\in[d]}$, receiver computes $u_0(x) = \prod_{i=1}^{d}(x - x_i)$.

2. Sender samples a $\mathsf{PRF}_k : \{0,1\}^* \to \{0,1\}^\kappa$ and compute $\ell'_B = \mathsf{PRF}_k(\ell)$.
For $Y = \{y_i\}_{i\in[d]}$, sender chooses a random degree -t polynomial f such that $f(0) = \ell'_B$ (f is defined by the secret label ℓ'_B from $\mathcal{F}^{d,t}_{\mathsf{TSSS}}$) then sender samples at random a degree-$2d$ polynomial $v_1(x) \in \mathbb{F}[x]$ such that $v_1(y_i) = f(y_i)$ for all $y_i \in Y$. Note that $\{v_1(y_i)\}_{i\in[d]}$ are then secret shares of ℓ'_B.

– **Online phase:**

1. Receiver and sender invoke to $\mathcal{F}^{1,d}_{\mathsf{vrOLE}}$ with inputs (InputR, u_0) and (InputS, v_1).
If $\mathcal{F}^{1,d}_{\mathsf{vrOLE}}$ aborts then aborts, otherwise sender gets $(\mathsf{OutputS}, u_1)$ and receiver gets $(\mathsf{OutputR}, v_0)$ such that $v_0 = u_0 \cdot u_1 + v_1$.
2. Sender sends $h_\ell = \mathcal{H}_1(\ell'_B)$ and $\Delta\ell = \mathcal{H}_2(\ell'_B) + \ell$ to receiver.
3. Receiver computes $V_0 := \{v_0(x_i)\}_{i\in[d]}$ and tries each combination of t elements in V_0 to recover ℓ'_A from $\mathcal{F}^{d,t}_{\mathsf{TSSS}}$.
4. Receiver checks if $h_\ell = \mathcal{H}_1(\ell'_A)$, then
   - recompute $\ell = \Delta\ell - \mathcal{H}_2(\ell'_A)$ and ℓ is the correct label of sender.
   - define the set $I = \{x_i | v_0(x_i)$ is a correct share of $\ell\}$ is $X \cap Y$.

   Otherwise, the output is $\perp$.

Security and Correctness.

Theorem 5. *Figure 9 securely realizes the ideal functionality $\mathcal{F}^{d,t}_{\mathsf{TLPSI}}$ against semi-honest adversaries in the $\mathcal{F}^{m,d}_{\mathsf{vrOLE}}$-hybrid model where $|\mathbb{F}| \geq 2^{\lambda+2\log d}$.*

Proof (Proof for Theorem 5). Our proof consists of two parts, correctness and security.

To guarantee the security we need to show that receiver learns nothing about the sender's input Y and the label l is kept secret. Since $f(0) = \ell'_B = \mathsf{PRF}_k(\ell)$ (k is kept secret by receiver) and if receiver learns ℓ'_B then it can reconstruct ℓ, we need to make sure v_0 reveals nothing about $f(0)$. We assume $0 \notin Y$ which means $\prod_{i=1}^{d}(0 - y_i) \cdot R(0)$ since the constant term in R is a random element. Note that $v_0(0) = u_0(0) \cdot u_1(0) + \ell'_B + \prod_{i=1}^{d}(0 - y_i) \cdot R(0)$ so ℓ'_B is hidden by the random element in $\mathbb{F}$. Secondly, since receiver's input to $\Pi^{d,t}_{\mathsf{TLPSI}}$ is the same as in $\mathcal{F}^{1,d}_{\mathsf{vrOLE}}$, Sim can extract actual input of $\mathcal{A}$ (corrupted receiver) and then interact with $\mathcal{A}$ to simulate the views of $\mathcal{A}$.

Correctness. Firstly, we show how to construct the polynomial v_1 of degree $2d$. We have $v_1(y_i) = f(y_i)$ for all $y_i \in Y$ so $\{y_i\}_{i\in[d]}$ are the root of the polynomial $v_1(x) - f(x)$, i.e., $v_1(x) - f(x) := \prod_{i=1}^{d}(x - y_i) \cdot R(x)$ where $R(x)$ is random polynomial of degree d over $\mathcal{R}$. The we can define the polynomial v_1 as below:

$$v_1(x) = f(x) + \prod_{i=1}^{d}(x - y_i) \cdot R(x)$$

Secondly, we show that the output of Figure 9 is correct. Given a label ℓ and two input sets X and Y, after receiver (receiver) and sender (sender) join $\mathcal{F}^{1,d}_{\mathsf{vrOLE}}$, receiver gets $v_0 = u_0 \cdot u_1 + v_1$.

For all $x_i \in X$, we have $u_0(x_i) = 0$, so receiver receives $v_0(x_i) = v_1(x_i)$. This means that if $x_i = y_i$ for some i, then a correct share $f(y_i)$ of ℓ'_B is recovered. receiver computes $V_0 := \{v_0(x_i)\}_{i\in[d]}$. If $X \cap Y \geq t$, then there are more than t correct secret shares in V_0, so receiver can find them by trying each combination of t elements in V_0 and recovers the correct label ℓ'_B. Therefore the set

$$I = \{x_i | v_0(x_i) \text{ is a correct share of } \ell'_B\}$$

is equal to $X \cap Y$ with high probability.

Given $x \in X \setminus Y$, observe that $v_0(x) = v_1(x) = f(x) + \prod_{i=1}^{d}(x - y_i) \cdot R(x) \neq f(x)$ with high probability, indeed since $x \in X \setminus Y \implies \prod_{i=1}^{d}(x - y_i) \neq 0$ and the probability of $R(x) = 0$ is at most $d/|\mathbb{F}|$ (the polynomial R is of degree d and random over $\mathcal{R}$), which means the probability that $v_0(x)$ is a correct share of ℓ, is negligible for $|\mathbb{F}| \geq 2^{\lambda+2\log d}$.

To reconstruct ℓ, receiver computes $V_0 := \{v_0(x_i)\}_{i\in[d]}$ and tries each combination of t elements in V_0 to recover ℓ'_A from $\mathcal{F}^{d,t}_{\mathsf{TSSS}}$. Receiver checks if $h_\ell = \mathcal{H}_1(\ell'_A)$, it means $\ell'_A = \ell'_B$ with high probability since $\mathcal{H}_1$ is collision-resistent hash of output $\{0,1\}^\kappa$. Therefore, receiver can reconstruct the label ℓ in plaintext as $\ell = \Delta\ell - \mathcal{H}_2(\ell'_A)$.

Security. It is easy to see that our protocol in Figure 9 is secure against a semi-honest sender. For a corrupted receiver, we prove our security in the hybrid model.

Sender is corrupted.

On input (Y, ℓ) as a sender to $\mathcal{F}_{\mathsf{TLPSI}}^{d,t}$ and get an output.
Since receiver does not send anything to sender except join the $\mathcal{F}_{\mathsf{vrOLE}}$ as a receiver with input u_0. By the security of $\mathcal{F}_{\mathsf{vrOLE}}$ that the sender gets no information about input of receiver so Sim acts as receiver and outputs whatever $\mathcal{F}_{\mathsf{TLPSI}}^{d,t}$ outputs. The view generated by running with Sim and the view by running with a receiver are then indistinguishable.

Receiver is corrupted. Sim interacts with $\mathcal{A}$ as below:

- Sim emulates $\mathcal{F}_{\mathsf{vrOLE}}^{1,d}$ and gets the input polynomial u_0 from $\mathcal{A}$. Sim defines a set X such that $X = \{x_i \mid u_0(x_i) = 0\}$. We have $|X| = d$.
- Sim takes the input set X of $\mathcal{A}$ and joins $\mathcal{F}_{\mathsf{TLPSI}}^{d,t}$ as a receiver and if $|X \cap Y| \geq t$, Sim learns $X \cap Y$ and label ℓ, otherwise learns nothing.
- If Sim gets $I := X \cap Y$ and label ℓ, Sim defines a set Y including of the set I and $n - |I|$ dummy items. Otherwise, Sim defines a set Y including of d dummy items y_i such that $y_i \notin X$ and a random label ℓ.
- Sim acts on behalf of sender with input Y and label ℓ.

Remarks about polynomials:

1. Given $\{(y_i, f(y_i))\}_{i\in[d]}$ as in the protocol, the polynomial v_1 is defined by $v_1(x) = f(x) + \prod_{i=1}^{d}(x - y_i) \cdot R(x)$ and $R(x)$ is uniformly random in $\mathcal{R}$, $f(x)$ is also random (except $f(0) = \mathsf{PRF}_k(\ell)$). Therefore, given a set $\{x_i\}_{i\in J}$ for $|J| \leq d$ and $x_i \notin \{y_i\}_{i\in[d]}$, by choosing at random v_1, the distribution of a set $\{v_1(x_i)\}_{i\in J}$ is uniformly random in $\mathbb{F}^d$.
2. Given $\{x_1, \ldots, x_d\}$ the polynomial v_0 as in the protocol reveals no more information beyond $\{v_1(x_1), \ldots, v_1(x_d)\}$ about v_1. Note that

$$v_0(x) = u_0(x) \cdot u_1(x) + v_1(x) = u_0(x) \cdot u_1(x) + f(x) + \prod_{i=1}^{d}(x - y_i) \cdot R(x)$$

is a $2d$-degree polynomial, where $u_1(x), R(x)$ are uniformly random polynomials of degree d, $2d$ over $\mathcal{R}$ respectively. We note that $f(x)$ is chosen randomly and only needs to satisfy $f(0) = \mathsf{PRF}_k(\ell)$. Therefore, from $v_0(x)$ receiver learns nothing about $\{y_i\}_{i\in[d]}$ and $f(0)$. Indeed, for any point $x' \notin \{x_i\}_{i\in[d]}$, we have $u_0(x') \neq 0$, so $v_0(x') = u_0(x') \cdot u_1(x') + f(x') + \prod_{i=1}^{d}(x' - y_i) \cdot R(x')$ is randomized by $u_1(x') \in \mathbb{F}, R(x') \in \mathbb{F}$ and $f(x')$ if $x' \neq 0$, consider $x' = 0$, $v_0(0) = u_0(0) \cdot u_1(0) + \ell'_B + \prod_{i=1}^{d}(0 - y_i) \cdot R(0)$, whp $\prod_{i=1}^{d}(0 - y_i) \cdot R(0) \neq 0$ then receiver learns nothing about $v_0(0)$ as well as $f(0)$. Therefore, given $\{x_i\}_{i\in[d]}$ (or u_0) as the input from corrupted receiver and $\{v_1(x_1), \ldots, v_1(x_d)\}$ as the output from the ideal world, the input polynomial v'_1 can be simulated by an interpolation at d points $\{(x_i, v_1(x_i))\}_{i\in[d]}$ and any $\{(x'_i, y'_i)\}_{i\in[d+1]}$ where $x'_i \notin \{x_i\}_{i\in[d]}$ and y'_i is uniformly random in $\mathbb{F}$. The indistinguishability is implied by the security of $\mathcal{F}_{\mathsf{vrOLE}}$ and the remark that we showed above.

The simulation in the case receiver is corrupted can not be distinguished from the real protocol by the following hybrids:

- Hyb 0. The same as real protocol, sender is honest with the input set Y, and $\mathcal{F}_{\mathsf{vrOLE}}$ is implemented honestly. By the security of $\mathcal{F}_{\mathsf{vrOLE}}$, the view of the adversary consists of $(h_\ell, \Delta\ell, u_0, v_0, \mathsf{out})$ where out is the output of the protocol.
- Hyb 1. Sim emulates $\mathcal{F}_{\mathsf{vrOLE}}^{d,\mathbb{F}}$ and gets the input polynomial u_0 from $\mathcal{A}$. Sim defines a set X such that $X = \{x_i \mid u_0(x_i) = 0\}$. Denote $|X| = d$. By the security of $\mathcal{F}_{\mathsf{vrOLE}}^{d,\mathbb{F}}$, this hybrid is identical to the previous hybrid.
- Hyb 2. Sim takes the input set X of $\mathcal{A}$ and joins $\mathcal{F}_{\mathsf{TLPSI}}^{d,t}$ as a receiver and if $|X \cap Y| \geq t$, Sim learns $X \cap Y$ and label ℓ otherwise, learns nothing. So depending on the output of $\mathcal{F}_{\mathsf{TLPSI}}^{d,t}$ we have two cases:

1. If Sim gets $I := X \cap Y$ and label ℓ then:

- Hyb 2-1. On behalf of sender, Sim sends $\mathcal{H}_1(\ell)$ to $\mathcal{A}$. There is no change in the adversary view.

- Hyb 2-2. Sim defines a set Y including of the set I and $d-|I|$ dummy items which do not belong to X. Sim then defines honestly the polynomial f with the input set Y and the label ℓ. There is no change in the distribution of f (defined by the secret sharing of $\mathsf{PRF}_k(\ell)$). Sim then takes at random a $2d$-degree polynomial v_1 such that if $y_i \in X \cap Y$, then $v_1(y_i) = f(y_i)$. We emphasize that by remark 1 the evaluation points $\{v_1(x_i)\}_{i\in[d]} := \{v_1(x_i) = f(y_i)\}_{x_i\in I} \cup \{v_1(x_i) \leftarrow\$ \mathbb{F}\}_{x_i\notin X\setminus I}$ with $|X \setminus I| \leq (d-t)$ have the same distribution as in real protocol.
- Hyb 1-3. Sim emulates $\mathcal{F}_{\mathsf{vrOLE}}$ on input $X = \{x_i\}_{i\in[d]}$ from the adversary and the simulated evaluation points $\{v_1(x_i)\}_{i\in[d]}$ as in remark 2. The simulated v_0 is then indistinguishable from the real protocol. The last component to analyze in the adversary view is the output of the protocol. In this hybrid, $\mathcal{A}$ gets a polynomial $v_0 = u_0 \cdot u_1 + v_1$, then for all $x_i \in X$:
  * If $x_i \in X \cap Y$ then $v_0(x_i) = v_1(x_i) = f(x_i)$ (correct share of ℓ'_B in $\mathcal{F}_{\mathsf{TSSS}}$).
  * If $x_i \notin X \cap Y$, then $v_0(x_i) = v_1(x_i) \neq f(x_i)$ with high probability since $v_1(x_i)$ is uniformly random in $\mathbb{F}$ (Hybrid 1-2). If we set $|\mathbb{F}|$ to be $2^{\lambda+2\log d}$, the probability that $v_1(x_i) = f(x_i)$ is negligible in λ. Therefore, the simulated output containing $(\ell, X \cap Y)$ is statistically the same as in the real protocol.

3. If $I := X \cap Y$ such that $|I| < t$, and Sim gets nothing then:
- Hyb 3-1. On behalf of sender, Sim picks randomly a label $\ell \in \mathbb{F}$, and a random $\ell'_B \leftarrow\$ \{0,1\}^\kappa$ and sends $\mathcal{H}_1(\ell'_B) \in \{0,1\}^\kappa, \Delta\ell = \mathcal{H}_2(\ell'_B) + \ell$ to $\mathcal{A}$. This hybrid is indistinguishable with the real protocol since the probability such that ℓ picked from Sim is the same as real label ℓ is negligible.
- Hyb 3-2. Sim defines a set Y including of d dummy items y_i such that $y_i \notin X$. Sim then defines honestly the polynomial f and v_1 with the input set Y and the label ℓ and value ℓ'_B. By remark 1, the simulated evaluation points $\{v_1(x_i)\}_{i\in[d]}$ are uniformly random in $\mathbb{F}^d$. We emphasize that in the real protocol even when $I \neq \emptyset$ and $|I| < t$, these evaluation points are also uniformly random. Indeed, we can separate these points in two sets: $\{v_1(x_i) = f(y_i)\}_{x_i\in I}$ and $\{v_1(x_i) \leftarrow\$ \mathbb{F}\}_{x_i\notin X\setminus I}$. By the security of $\mathcal{F}_{\mathsf{TSSS}}$, the set of correct shares $\{f(y_i)\}_{x_i\in I}$ are uniformly random as $|I| < t$. The set $\{v_1(x_i) \leftarrow\$ \mathbb{F}\}_{x_i\notin X\setminus I}$ are uniformly random by remark 1.
- Hyb 3-3. Sim emulates $\mathcal{F}_{\mathsf{vrOLE}}$ on input $X = \{x_i\}_{i\in[d]}$ from the adversary and the simulated evaluation points $\{v_1(x_i)\}_{i\in[d]}$ as in remark 2. The simulated v_0 is then indistinguishable from the real protocol. The last component to analyze from the adversary's point of view is the output of the protocol. Similarly to the first case, if $x_i \notin X \cap Y$, then $v_0(x_i) = v_1(x_i) = f(x_i)$ with a negligible probability. The probability that the correct label ℓ'_B is recovered is thus negligible. Therefore, the simulated output is random (containing no information) as in the real protocol.

5.2 Fuzzy-labeled PSI

With the masked OPRF $\mathcal{F}_{\mathsf{mOPRF}}$ functionality and the threshold-labeled PSI $\mathcal{F}_{\mathsf{TLPSI}}$ functionality, we are ready to describe our fuzzy-labeled PSI protocol. The first step is to describe the sub-sampling procedure and analyze its error probability.

5.3 Sub-sampling

The sub-sampling method from [UCK+21] is adapted to our work to construct fuzzy PSI from threshold-labeled PSI. We give a thorough analysis of the false positive rate and the false negative rate which was not present in [UCK+21]. The core building block is a sub-sampling technique that works as follows. Suppose receiver has input $x \in \{0,1\}^d$ and sender has input $y_i \in \{0,1\}^d$ for $i \in [m]$ (ignoring the label for now). For every binary vector of length d the parties map it to T vectors of length d_s, where $d_s < d$. That is, $x \mapsto \{x_1, \ldots, x_T\}$, $y_i \mapsto \{y_{i,1}, \ldots, y_{i,T}\}$ and $|y_{i,j}| = d_s$. If x and y_i are close in Hamming distance d_k, then $|\bar{X} \cap \bar{Y}_i| \geq t$. where $\bar{X} := \{x_1, \ldots, x_T\}$ and $\bar{Y}_i := \{y_{i,1}, \ldots, y_{i,T}\}$

In practice, elements in $\bar{X}$ and $\bar{Y}_i$ are computed by applying x and y_i with T different masks $\{m_i\}_{i\in[T]}$, such that $\mathsf{HW}(m_i) = d_s$. Namely, $\bar{X} \leftarrow \{x \wedge m_1, \ldots, x \wedge m_T\}$ and $\bar{Y}_i \leftarrow \{y \wedge m_1, \ldots, y \wedge m_T\}$.

We need to select parameters t, T and d_s such that if $\mathsf{HW}(x \oplus y) \leq d_k$, then $(\bar{X} \cap \bar{Y}_i) \geq t$ with a low false positive rate FPR and a low false negative rate FNR.

Let $x \in \{0,1\}^d$ and $y \in \{0,1\}^d$, which are receiver's and sender's inputs (assuming sender has only one input), respectively. Recall that we have $\{m_i\}_{i\in[T]}$ masks which have Hamming weight of

d_s. For a vector $x \in \{0,1\}^d$ with Hamming weight of l we denote a tuple $(h_1^x, \ldots, h_l^x)$ as the position in the vector x having bit value 1. We denote the event $\mathcal{S}_v$ as the event where $(x \wedge m_i = y \wedge m_i)$ given $\mathsf{HW}(x \oplus y) = v$. Indeed, for each $i \in [1, T]$:

$$\begin{aligned}\Pr_{m_i \leftarrow \$ \mathcal{B}_{d_s}}[\mathcal{S}_v] &= \Pr_{m_i \leftarrow \$ \mathcal{B}_{d_s}}\left[\{h_j^{m_i}\}_{j\in[d_s]} \cap \{h_j^{x\oplus y}\}_{j\in[v]} = \emptyset\right] \\ &= \prod_{i=0}^{d_s-1}\left(\frac{d-v-i}{d}\right),\end{aligned}$$

where $m_i \leftarrow\$\ \mathcal{B}_{d_s}$ denotes sampling a length d bit-string with Hamming weight d_s. In other words, this is the probability of sampling a vector of length d having d_s 1s such that the positions of all 1s do not overlap any of the v 1s in the vector $x \oplus y$. If the overlapping happens then $(x \wedge m_i \neq y \wedge m_i)$.

FNR and FPR The FNR is the probability that the outcome is "far" when the Hamming distance between the two inputs x and y are "close" with respective to some threshold d_k, i.e., $\mathsf{HW}(x \oplus y) \leq d_k$. Using the binomial, the FNR from sub-sampling is given by

$$\sum_{v=0}^{d_k}\left(\Pr[\mathsf{HW}(x\oplus y) = v] \cdot \left(\sum_{c=0}^{t-1}\binom{T}{c} \cdot \Pr[\mathcal{S}_v]^c \cdot (1-\Pr[\mathcal{S}_v])^{T-c}\right)\right).$$

The binomial term inside this is the probability where the event $\Pr[\mathcal{S}_v]$ occurred less than t-out-of-T times, i.e., less than t-out-of-T of the sub-samples will match. Additionally, $v \leq d_k$ implies the true outcome is "close". Multiplying the binomial term with $\Pr[\mathsf{HW}(x \oplus y) = v]$, and then taking the sum gives us the FNR.

The FPR is the probability that the outcome is "close" when the Hamming distance is "far", which can be computed in a similar way:

$$\begin{aligned}\sum_{v=d_k+1}^{d-d_s}\Bigg(&\Pr[\mathsf{HW}(x\oplus y) = v]\cdot \\ &\left(1 - \sum_{c=0}^{t-1}\binom{T}{c} \cdot \Pr[\mathcal{S}_v]^c \cdot (1-\Pr[\mathcal{S}_v])^{T-c}\right)\Bigg).\end{aligned} \tag{1}$$

The difference from the FNR is that v is a sequence in $[d_k + 1, d - d_s]$, which implies the distance is above the threshold d_k and ends with $d - d_s$ since it is not possible to have a false positive where the Hamming distance is more than $d - d_s$ and d_s unique bits are sampled. Also the binomial term inside suggests a "close" outcome since t or more sub-samples will match.

Hamming Distance Probability Finally, we need to discuss the Hamming distance probability $\Pr[\mathsf{HW}(x \oplus y) = v]$. Suppose x and y are sampled uniformly at random, then the number of bits that differ follows the binomial distribution and for every bit in $x \oplus y$ the probability that the bit is 1 is $1/2$. Thus we have

$$\Pr[\mathsf{HW}(x\oplus y) = v] = \binom{d}{v} \cdot \left(\frac{1}{2}\right)^v \cdot \left(\frac{1}{2}\right)^{d-v} = \frac{1}{2^d} \cdot \binom{d}{v}.$$

Truncation As we will see later, the masked output is firstly permuted using a PRF and then, optionally, truncated to fewer than d bits. The truncation step will introduce additional error which we analyze below.

Consider two elements x and y with their corresponding truncated version x' and y'. The FNR does not increase because the probability that $x' \neq y'$ after truncation when they were initially the same $(x = y)$ is not possible. However, it may decrease since the truncation may "correct" a false negative. In more detail, a false negative happens when there are less than t-out-of-T (full-length) matches when two elements are actually close, but truncation increases the chance for a match to happen. Thus the overall FNR can be upper bounded by the sub-sampling FNR:

$$\mathsf{FNR} \leq \sum_{v=0}^{d_k}\left(\frac{1}{2^d} \cdot \binom{d}{v} \cdot \left(\sum_{c=0}^{t-1}\binom{T}{c} \cdot \Pr[\mathcal{S}_v]^c \cdot (1-\Pr[\mathcal{S}_v])^{T-c}\right)\right).$$

On the other hand, the FPR will increase. So we need to consider the extra probability of getting two of the same truncated output ($x' = y'$) when they were initially different ($x \neq y$), in addition to the FPR from sub-sampling. Concretely, for $|x| = d$ and $|x'| = k$ we have $\Pr[x' = y' \mid x \neq y] \approx 1/2^k$ where we assume $\Pr[x \neq y] \approx 1$ for large d. The overall FPR we introduce in truncation is when at least one of such even happening out of T, i.e., $1-(1-(1/2^k))^T$. Similarly, we can bound the overall FPR as follows

$$\mathsf{FPR} \leq \left(1 - \left(1 - \frac{1}{2^k}\right)^T\right) + \sum_{v=d_k+1}^{d-d_s} \left(\frac{1}{2^d} \cdot \binom{d}{v} \cdot \left(1 - \sum_{c=0}^{t-1} \binom{T}{c} \cdot \Pr[\mathcal{S}_v]^c \cdot (1 - \Pr[\mathcal{S}_v])^{T-c}\right)\right).$$

Sub-sampling. The sub-sampling method from [UCK+21] is adapted to our work to construct fuzzy PSI from threshold-labeled PSI that works as follows. Suppose receiver has input $x \in \{0,1\}^d$ and sender has input $y_i \in \{0,1\}^d$ for $i \in [m]$ (ignoring the label for now). For every binary vector of length d the parties map it to T vectors of length d_s, where $d_s < d$. That is, $x \mapsto \{x_1, \ldots, x_T\}$, $y_i \mapsto \{y_{i,1}, \ldots, y_{i,T}\}$ and $|y_{i,j}| = d_s$. If x and y_i are close in Hamming distance d_k, then $|\bar{X} \cap \bar{Y}_i| \geq t$. where $\bar{X} := \{x_1, \ldots, x_T\}$ and $\bar{Y}_i := \{y_{i,1}, \ldots, y_{i,T}\}$

In practice, elements in $\bar{X}$ and $\bar{Y}_i$ are computed by applying x and y_i with T different masks $\{m_i\}_{i\in[T]}$, such that $\mathsf{HW}(m_i) = d_s$. Namely, $\bar{X} \leftarrow \{x \wedge m_1, \ldots, x \wedge m_T\}$ and $\bar{Y}_i \leftarrow \{y \wedge m_1, \ldots, y \wedge m_T\}$. Just like the work of [UCK+21], this approach inherently introduces false negative and false positives we analyze the probabilities in detail in Section 5.3. We select parameters t, T and d_s such that if $\mathsf{HW}(x \oplus y) \leq d_k$, then $(\bar{X} \cap \bar{Y}_i) \geq t$ with a low false positive rate FPR and a low false negative rate FNR (discussed in Section 6.2).

Masked OPRF. The sub-sampling procedure above does not discuss a concrete protocol for obtaining the masks and performing the sub-sampling. Consider the naive case where receiver and sender agree on the masks, perform OPRF on the masked inputs (to protect sender's privacy), and then use TLPSI to compute the final label. Since the sub-sampling is public, this protocol leaks the d_s sub-sampled plaintext bits of sender's input $y_i \in \{0,1\}^d$ if there is an intersection above the threshold t.

To avoid the leakage issue above, receiver and sender need to run a modified OPRF protocol, which we call masked OPRF. In masked-OPRF, the input of receiver is her original input $x \in \{0,1\}^d$ instead of the sub-sampled input. sender additionally inputs masks $\{m_j : j \in [T]\}$, where every mask has Hamming weight d_s. The output of receiver becomes $X' := \{\mathsf{PRF}_k(x \wedge m_1), \ldots, \mathsf{PRF}_k(x \wedge m_T)\}$ (without knowing the PRF key k). sender performs the same except his input is also masked: $Y'_i := \{\mathsf{PRF}_k(y_i \wedge m_1), \ldots, \mathsf{PRF}_k(y_i \wedge m_T)\}$ for $i \in [m]$. The functionality can be found in Figure 4.

We note that the communication cost of masked OPRF is only dependent on d ($d \ll m$) since sender holds both PRF key and the set of masks, hence sender can *local* compute the masked PRF output. In addition, the instantiation of masked OPRF is straightforward if a garbled circuit based OPRF is given. Creating the masks are trivial in this setting since, if the mask is 0, the receiver (receiver) can simply set both of the evaluator's input labels to the zero-label as input to the oblivious transfer. If the mask is 1, the input labels are unchanged.

Instantiation. Our fuzzy-labeled PSI protocol consists of two steps (1) the parties run sub-sampling using masked OPRF $\mathcal{F}_{\mathsf{mOPRF}}$ to obtain X' and $\{Y'_i\}_{i\in[m]}$ and (2) using X' and $\{Y'_i\}_{i\in[m]}$ as input to the threshold-labeled PSI functionality $\mathcal{F}_{\mathsf{TLPSI}}$ to obtain labels for the corresponding matching elements, if any. Since X' and $\{Y'_i\}_{i\in[m]}$ are obtained from subsampling, if x and y_i are close then the size of intersection of X' and Y'_i are always larger than a small t, which enables us to use our TLPSI construction. We note that $\{Y'_i\}_{i\in[m]}$ is output of masked OPRF then with high probability $0 \notin Y'_i$. The protocol is described in Figure 10, and the security proof in a semi-honest setting is given Theorem 6.

Fig. 10: Π_{FLPSI} **from** $\mathcal{F}_{\mathsf{TLPSI}}^{T,t}$ **and** $\mathcal{F}_{\mathsf{mOPRF}}^{T}$.

PARAMETERS AND INPUTS:

- Receiver has input $x \in \{0,1\}^d$ while sender has a set of inputs $Y := \{(y_i, \ell_i)\}_{i\in[m]}$ where $y_i \in \{0,1\}^d, \ell_i \in \mathbb{F}$.
- A Hamming distance with a threshold parameter $d_k \ll d$ having TPR and TNR.
- A t-out-of-T threshold-labeled PSI protocol $\mathcal{F}_{\mathsf{TLPSI}}^{T,t}$ in Figure 9 over $\mathcal{R}$ where T is the number of sub-samples.
- A masked-OPRF $\mathcal{F}_{\mathsf{mOPRF}}$ instantiated using a pseudorandom function $\mathsf{PRF}_k(m) : \{0,1\}^\kappa \times \{0,1\}^\kappa \to \{0,1\}^\kappa$ that takes input a key $k \in \{0,1\}^\kappa$ and message $m \in \{0,1\}^\kappa$.

PROTOCOL:

1. Sender samples uniformly a PRF key k and masks $\{m_1, \ldots, m_T\}$, each mask has Hamming weight d_s where $d_s < d_k$ (see Section 6.2 for concrete choice of d_s).
2. Receiver and sender invoke to $\mathcal{F}_{\mathsf{mOPRF}}$ with input (InputR, x) and $(\mathsf{InputS}, k, \{m_1, \ldots, m_T\})$. At the end of the protocol, receiver obtains
$$X' := \{\mathsf{PRF}_k(x \wedge m_1), \ldots, \mathsf{PRF}_k(x \wedge m_T)\}.$$
3. Sender executes PRF on his input to obtain
$$\{Y_i' := \{\mathsf{PRF}_k(y_i \wedge m_1), \ldots, \mathsf{PRF}_k(y_i \wedge m_T)\} : i \in [m]\}.$$
4. Add a tweak where receiver's input to $\mathcal{F}_{\mathsf{TLPSI}}^{T,t}$ becomes $X' := \{X_1' \oplus 1, X_2' \oplus 2, \ldots, X_T' \oplus T\}$ and sender's input $(Y_i')_{i\in[m]}$ is tweaked in the same way.
5. For $i \in [m]$, receiver inputs (InputR, X') and sender inputs $(\mathsf{InputS}, Y_i', \ell_i)$ to $\mathcal{F}_{\mathsf{TLPSI}}^{T,t}$ then:
   - Receiver obtains ℓ_i if $|X' \cap Y_i'| \geq t$,
   - Otherwise receiver learns $\perp$.

   // need only one instance of $\mathcal{F}_{\mathsf{vrOLE}}^{m,T}$ to instantiate m times of $\mathcal{F}_{\mathsf{TLPSI}}^{T,t}$.
6. The leakage profile is $\mathcal{L} := \{\mathcal{L}_{\mathbf{R}}, \mathcal{L}_{\mathbf{S}}\}$ such that $\mathcal{L}_{\mathbf{R}} = \{X' \cap Y_i' : |X' \cap Y_i'| \geq t\}_{i\in[m]}$ and $\mathcal{L}_{\mathbf{S}} = \perp$.

A subtle issue may occur depending on the sub-sampling parameters. Specifically, there might be duplicates in X', or one or more Y_i', especially when T is large or d_s is small. In the extreme case, if $d_s = 1$, then the probability that two sub-samples will match is $1/2$. This can be resolved by adding a tweak to every output of the PRF, which we specify in step 4 of Figure 10. For more details about leakage profile $\mathcal{L}$, in our construction, sender learns nothing $\mathcal{L}_{\mathsf{S}} = \perp$ and receiver learns the label ℓ_i (actually output) with overwhelming probability (at least TPR) if $\mathsf{HW}(x \oplus y_i) < t$, receiver also learns extra information $\mathcal{L}_{\mathsf{R}}$, which is the set of (obviously) encrypted sub-samplings that matched. However, we emphasize that this leakage to the receiver is strictly less than the receiver learning the matching entries of the sender's database, because of the masked OPRF, which helps to obfuscate the actual sender's database.

Theorem 6. *The protocol with the leakage profile $\mathcal{L}$ in Figure 10 is correct and securely realizes the ideal functionality $\mathcal{F}_{\mathsf{FLPSI}}$ against a semi-honest setting in the $\{\mathcal{F}_{\mathsf{TLPSI}}^{T,t}, \mathcal{F}_{\mathsf{mOPRF}}^{T}\}$-hybrid models.*

Proof (Proof for Theorem 6). Our proof consists of two parts, correctness and security.

Correctness. By the correctness of $\mathcal{F}_{\mathsf{TLPSI}}$, receiver obtains ℓ_i if $|X' \cap Y_i'| \geq t$. With our parameters, the TPR and TFR can be kept as high as $1 - 7 \times 10^{-6}$ and $1 - 2 \times 10^{-16}$, respectively.

Security. We prove that our construction is secure against semi-honest adversaries in $\{\mathcal{F}_{\mathsf{TLPSI}}^{T,t}, \mathcal{F}_{\mathsf{mOPRF}}^{T}\}$-hybrid models. We construct simulators Sim that can simulate the view of $\mathcal{A}$ by the following hybrid games.

Sender is corrupted. Sim interacts with $\mathcal{A}$ as below:

- Since there is no interaction between sender and receiver except they invoke into $\{\mathcal{F}_{\mathsf{TLPSI}}^{T,t}, \mathcal{F}_{\mathsf{mOPRF}}^{T}\}$, therefore, Sim with input $Y = \{(y_i, \ell_i)\}_{i\in[m]}$ of $\mathcal{A}$ can simulate $\mathcal{A}$'s views as in real execution by emulating $\mathcal{F}_{\mathsf{TLPSI}}^{T,t}$ and $\mathcal{F}_{\mathsf{mOPRF}}^{T}$.
- Sim inputs $Y = \{(y_i, \ell_i)\}_{i\in[m]}$ to FLPSI as a sender and gets the output, Sim emulates $\mathcal{F}_{\mathsf{TLPSI}}^{T,t}$ to output whatever $\mathcal{F}_{\mathsf{FLPSI}}$ outputs.

The simulation against corrupted sender is indistinguishable from the real protocol by the following hybrids:

- Hyb 0. The same as real protocol, the receiver is honest with the input $x \in \{0,1\}^d$, and $\mathcal{F}_{\mathsf{TLPSI}}^{T,t}, \mathcal{F}_{\mathsf{mOPRF}}^{T}$ are instantiated honestly.
- Hyb 1. Sim emulates $\mathcal{F}_{\mathsf{mOPRF}}^{T}$ as a receiver. Specifically, in step 2, by the security of $\mathcal{F}_{\mathsf{mOPRF}}^{T}$ that the sender gets no information, so Sim can emulate the instantiation of $\mathcal{F}_{\mathsf{mOPRF}}^{T}$ on input Y, a key k and T sub-samples $\{m_1, \ldots, m_T\}$.
  This hybrid is indistinguishable from the Hyb 0 since $\mathcal{A}$ does not learn anything after executing the instantiation of $\mathcal{F}_{\mathsf{mOPRF}}^{T}$ that is securely realized the ideal functionality of $\mathcal{F}_{\mathsf{mOPRF}}^{T}$.
- Hyb 2. Sim inputs $Y = \{(y_i, \ell_i)\}_{i\in[m]}$ to FLPSI as a sender and gets the output. This hybrid is identical to the previous one.
- Hyb 3. In step 5, Sim can emulate the instantiation of $\mathcal{F}_{\mathsf{TLPSI}}^{T,t}$ on the simulated key k and the simulated set $\{Y_i' = (y_i \wedge m_1, \ldots, y_i \wedge m_T)\}_{i\in[m]}$, and output whatever FLPSI outputs in Hyb 2. This hybrid is indistinguishable from the Hyb 3 since the instantiation of $\mathcal{F}_{\mathsf{TLPSI}}^{T,t}$ is securely realized $\mathcal{F}_{\mathsf{TLPSI}}^{T,t}$ ideal functionality.

Receiver is corrupted. Sim interacts with $\mathcal{A}$ as below:

- Sim with $\mathcal{A}$'s input x joins $\mathcal{F}_{\mathsf{FLPSI}}$ and gets the output set $R = \{\ell_i : |X' \cap Y_i'| \geq t\}_{i\in[m]}$ and the leakage $\mathcal{L}_{\mathsf{R}} = \{X' \cap Y_i' : |X' \cap Y_i'| \geq t\}_{i\in[m]}$. Sim has to simulate honest sender in the communication with receiver in step 2 and step 5.
- In step 2, for each $i \in [T]$, it assigns a random value $r_i \in \{0,1\}^\lambda \setminus \{0\}$ to $\mathsf{PRF}_k(x \wedge m_i)$. If $m_i = m_j$ for some $i \neq j$, then $r_i = r_j$. From the simulated output $\{r_i\}_{i\in[T]}$ and receiver's input x, Sim emulates the instantiation of $\mathcal{F}_{\mathsf{mOPRF}}^{T}$. Sim emulates $\mathcal{F}_{\mathsf{mOPRF}}^{T}$.
- In step 5, based on the output set R and the leakage $\mathcal{L}_{\mathsf{R}}$, Sim simulates the output of $\mathcal{F}_{\mathsf{TLPSI}}$ as follows:
  - For any $i \in [m]$ such that $\ell_i \in R$ (match between x and y_i), the output for the i-th instance of $\mathcal{F}_{\mathsf{TLPSI}}$ is set to be $R_{\mathsf{Sim},i} = (\ell_i, I_i)$ where
  $$I_i = \{r_j | j \text{ is a position where } X' \text{ and } Y_i' \text{ intersect}\}_{j\in[T]}.$$
  - For any $i \in [m]$ such that $\ell_i \notin R$ (no match between x and y_i), the output for the i-th instance of $\mathcal{F}_{\mathsf{TLPSI}}$ is set to $R_{\mathsf{Sim},i} = \perp$.
- For each $i \in [m]$, Sim emulates the instantiation of $\mathcal{F}_{\mathsf{TLPSI}}$ from receiver's simulated input $\{r_i\}_{i\in[T]}$ and the simulated output $R_{\mathsf{Sim},i}$.

The simulation against corrupted receiver is indistinguishable from the real protocol by the following hybrids:

- Hyb 0. The same as real protocol, sender is honest with the input set Y. The view of the adversary consists of the $\mathcal{F}_{\mathsf{mOPRF}}^{T}$ output $X' = \{\mathsf{PRF}_k(x \wedge m_i)\}_{i\in[T]}$, the $\mathcal{F}_{\mathsf{TLPSI}}$ outputs $R = \{R_i\}_{i\in[m]}$ and the leakage $\mathcal{L}_{\mathsf{R}}$.
- Hyb 1. As in the previous hybrid, except that the $\mathcal{F}_{\mathsf{mOPRF}}^{T}$ output $X' = \{r_i\}_{i\in[T]}$ for uniformly random $r_i \in \{0,1\}^\lambda \setminus \{0\}$. By the security of PRF, replacing each $\mathsf{PRF}_k(x \wedge m_i)$ by r_i is indistinguishable. Since T is $O(\lambda)$, then replacing T instances of PRF is still indistinguishable.
- Hyb 2. As in the previous hybrid, except that the execution of $\mathcal{F}_{\mathsf{mOPRF}}^{T}$ is emulated by Sim on input x and $X' = \{r_i\}_{i\in[T]}$. The indistinguishability comes from the security of $\mathcal{F}_{\mathsf{mOPRF}}^{T}$.
- Hyb 3. This is the same as the hybrid above except that the execution of $\mathcal{F}_{\mathsf{TLPSI}}$ is emulated by Sim on input $X' = \{r_i\}_{i\in[T]}$ and $R_{\mathsf{Sim},i}$ (as described above). In each $R_{\mathsf{Sim},i}$, by using the leakage from the real protocol, the distribution of positions where X' and Y_i' intersect is identical to that in the real protocol. Therefore, the distribution of $\{R_{\mathsf{Sim},i}\}_{i\in[m]}$ (which contains $\mathcal{F}_{\mathsf{TLPSI}}$ outputs and leakage to receiver) is identical to the output and the leakage of m instances of $\mathcal{F}_{\mathsf{TLPSI}}$ in the real protocol. By the fact that $r_i \neq 0$ for all $i \in [T]$, the security of $\mathcal{F}_{\mathsf{TLPSI}}$ holds. The indistinguishability between Hyb 2 and Hyb 3 is then implied directly from the emulation of m instances of $\mathcal{F}_{\mathsf{TLPSI}}$ where $m = \mathsf{poly}(\lambda)$.

6 Evaluation

6.1 Theoretical Comparison

We compare our fuzzy-labeled PSI with the state-of-the-art protocol [UCK+21] which builds an efficient threshold set-matching protocol from leveled homomorphic encryption, garbled circuits (GC),

and t-out-of-T secret sharing. However, we found that the parameter selection of [UCK+21] somewhat fails to meet the functionality requirement, which we discuss in detail in Section 6.3. Since our scheme is based on vector OLE, we also compare our work in terms of performance with the work of Chakraborti et al. [CFR23] that uses OLE as well, even though their work does not support labels. In particular, [CFR23] presents a fuzzy PSI protocol for Hamming distance based on additively homomorphic encryption and vector oblivious linear evaluation (VOLE).

Additionally, we give an asymptotic comparison with PCG-based ring OLE PSI [CILO22] and fuzzy PSIs [GRS22, vBP24, GQL+24] that support different distance metrics than us. The protocols described in [CILO22, GRS22] do not support labels and [vBP24, GQL+24] supports labels but it performs well due to its reliance on ElGamal encryption and additive homomorphic encryption (AHE). While [CILO22] uses ring-OLE instead of VOLE and then obtains efficient PSI protocol variants; Garimella, Rosulek and Singh [GRS22] supports distance L_∞ and L_1 with a large receiver's computation, later [vBP24] overcomes this limit by proposing fuzzy PSI based on DDH that can support L_∞ and L_p for $p \in [1, \infty)$, however the receiver's computation and communication cost is still $O(2^d \cdot N)$ leaving a poor performance when dimension d is large.

The asymptotic communication comparison is shown in Table 2. We ignore the cost of setting up masked OPRF since all protocols uses the same masked OPRF and the cost for this is independent to the dataset size. The dominant term $O(T\lambda M)$ in our online cost comes from de-randomizing VOLE to obtain vector ring-OLE over the field $\mathbb{F}$ where $|\mathbb{F}| \geq 2^{\lambda + 2\log d}$. Our offline communication cost is $O(\kappa)$ as the cost for generating VOLE (omitted the minor logarithmic on length of VOLE term). Compared to [UCK+21, CFR23], our online communication cost does not depend on the computational security parameter κ. We support an arbitrarily large label size which equals the bit-length of field $\mathbb{F}$ and in the same setting, our FPR and FNR are both negligible.

6.2 Concrete Comparison

Implementation. We provide an open source implementation[6] of our fuzzy PSI protocol using the swanky [Gal19] framework in the Rust programming language.

Our masked OPRF is based on evaluating fixed-key AES-128 using garbled circuits. The specific instantiation of the masked OPRF does not affect the communication or computation cost significantly since the cost does not depend on the size of sender's database m. For sub-sampling, the specific parameters we use are $d = 128$, $T = 64$, $d_s = 14$ and $t = 2$ which achieves a FPR of less than 7×10^{-6} and a FNR of less than 2×10^{-16} for $d_k \leq 20$.

Next, we perform semi-honest threshold labeled PSI (Figure 9), which we implement using VOLE in $\mathbb{F}_p$ where $p = 2^{61} - 1$. This is possible since we can simply truncate the mOPRF output to 61 bits. Note that this field size is much higher than what is used in Uzun et al. [UCK+21], which is only 23 bits due to their FHE parameterization[7]. The effect of truncation on the FPR in our scheme is negligible and the statement above remains true. Together, our parameters guarantee $\kappa = 128$ and $\lambda = 40$.

Performance. All the experiments are performed on GNU/Linux machines running on Intel(R) Core(TM) i9-9900 CPU @ 3.10GHz. The end-to-end latency (on LAN) and communication costs can be found in Table 3. Similar to Uzun et al. [UCK+21], we consider the steps that do not depend on the receiver input to be in the preprocessing step. This includes generating the VOLE correlations, garbling the AES-128 circuit, and so on. Our protocol has slightly better end-to-end latency while using less than 30% of their communication and supporting a much higher label size at the same time. We remark that the protocol from [CFR23] has similar performance characteristics as our protocol but it does not support labels. Note that since neither work has an open source implementation, we only compare with values reported in the papers.

[6] `https://github.com/kc1212/flash-psi`

[7] From our understanding, the parameters used in Uzun et al. [UCK+21] cannot support labels. See Section 6.3 for details.

[8] The overhead over L_1 is $2^d/d$ larger than over L_∞ so we take the cost over L_∞ for setting when the receiver's points are distance $> 4\delta$ apart and factor $2^d/d$ times.

[9] The receiver's points are distance $> 2\delta(d+1)$ apart.

Table 2. We use M and N to denote the input sizes of the sender and the receiver, respectively, where $M \gg N$. T is number of sub-sampling vectors, λ, κ are statistical and computational parameters. For HE-based protocols [UCK+21, CFR23] We use our existing notation except c and B are HE parameters for the number of slots and the number of partitions, respectively. Logarithmic terms are hidden in $\tilde{O}$. For [GRS22, vBP24, GQL+24], sender and receiver's input are hyperballs of radius δ with dimension $d \ll \kappa$, we cast their setting in L_1 distance while others are in Hamming distance.

Protocol	Communication	Technique	Distance	Label
[UCK+21] online [UCK+21] offline	$\tilde{O}(\kappa \cdot \frac{MT}{cB})$ $\tilde{O}(\kappa)$	HE	Hamming (dim. κ)	not support (Section 6.3
[CILO22] online [CILO22] offline	$O(\lambda M)$ $O(\lambda M)$	ring-OLE	✗	✗
[GRS22]	$O(\delta 4^d N + (2^d/d)M)$[8]	OT	L_1 (dim. d)	✗
[CFR23] online [CFR23] offline	$O(T\kappa M)$ $O(\kappa)$	AHE, OLE	Hamming (dim. κ)	✗
[vBP24]	$O(\delta 2^d dN + \delta M)$[9]	DDH	L_1 (dim. d)	yes
[GQL+24]	$O(d^2 M + \delta dN)$	AHE DDH	Hamming (dim. d)	yes
ours online ours offline	$O(T\lambda M)$ $O(\kappa)$	VOLE	Hamming (dim. κ)	yes with size $\log_2 \lvert\mathbb{F}\rvert$

Table 3. End-to-end latency and communication cost of our fuzzy PSI protocol on a single thread (Figure 10), where m is the set size of sender.

m	end-to-end latency (s)			communication (MB)		
	ours	[UCK+21]	[CFR23]	ours	[UCK+21]	[CFR23]
$10,000$	1.69	2.12	2.00	15.5	72	25.8
$100,000$	16.5	17.8	11.0	153	528	220
$1,000,000$	167	189	130	1533	2124	1504

6.3 Label Length of Uzun et al. [UCK+21]

We briefly describe the way FHE-based TLPSI from [UCK+21] works as follows. The server inputs labels $\{l_i\}_{i\in[m]}$, which are padded by zeros to become $(0^\lambda \| l_i)$ and then secret shared using a t-out-of-T secret sharing scheme and we denote the shares using $\{s_{i,1}, \ldots, s_{i,T}\}$. The reason that the labels are padded is because later, in the reconstruction phase, the client can identify whether the reconstruction is successful by checking whether the padding is correctly formed. The server also inputs items of the form $Y_i := \{Y_{i,1}, \ldots, Y_{i,T}\}$ and the corresponding label l_i should be returned to the client if $|X \cup Y_i| \geq t$. This problem can be solved using an unbalanced labeled PSI [CHLR18, CMdG+21], where the labels in this case are the shares $s_{i,j}$.

In this class of protocols, it is common to construct intersection polynomials such that $P(Y_{i,j}) = s_{i,j}$ and then partition the coefficients of P to reduce the multiplicative depth of polynomial evaluation, which allows better parameterization in the underlying FHE scheme. For the detailed partitioning and windowing technique, we refer to [UCK+21, Section 5.4.2] and [CHLR18]. In summary, the authors construct plaintext polynomials p_k^ℓ for $\ell \in [B]$ and $k \in [a]$ where each polynomial has m slots and B and a can be considered as the partition parameters. Finally $m \cdot a$ evaluation results, which may or may not be one of the shares $s_{i,j}$, are returned to the client and then one or more labels are returned by attempting to reconstruct $\binom{T}{t}$ results.

One set of FHE parameters is provided in [UCK+21] for all database length m. The plaintext polynomial has degree $m_p = 2^{13}$ and coefficients in modulo $p = 8519681$, which is split into 8192 factors of degree 1. Since p has 23-bits, the construction encodes each share $s_{i,j}$ into one slot of length 23-bits. Recall that the shares are constructed from $(0^\lambda \| l_i)$, and λ is also selected to be 23-bits

(see [UCK+21, Figure 6]), this leaves us with label sizes of 0. Although the scheme supports labels, the parameters used in [UCK+21] cannot support it to the best of our knowledge.

Acknowledgment

Dung Bui is supported by the funding from the European Union's Horizon Europe research and innovation programme under the project "Quantum Security Networks Partnership" (QSNP, grant agreement No 101114043). This work was done partially when both authors were at COSIC, KU Leuven (Belgium).

References

ABD+21. Navid Alamati, Pedro Branco, Nico Döttling, Sanjam Garg, Mohammad Hajiabadi, and Sihang Pu. Laconic private set intersection and applications. In Kobbi Nissim and Brent Waters, editors, *TCC 2021, Part III*, volume 13044 of *LNCS*, pages 94–125. Springer, Heidelberg, November 2021. [-]

ADI+17. Benny Applebaum, Ivan Damgård, Yuval Ishai, Michael Nielsen, and Lior Zichron. Secure arithmetic computation with constant computational overhead. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 223–254. Springer, Heidelberg, August 2017. [-]

ADT11. Giuseppe Ateniese, Emiliano De Cristofaro, and Gene Tsudik. (If) size matters: Size-hiding private set intersection. In Dario Catalano, Nelly Fazio, Rosario Gennaro, and Antonio Nicolosi, editors, *PKC 2011*, volume 6571 of *LNCS*, pages 156–173. Springer, Heidelberg, March 2011. [-]

ALOS22. Diego F. Aranha, Chuanwei Lin, Claudio Orlandi, and Mark Simkin. Laconic private set-intersection from pairings. In Yin et al. [YSCS22], pages 111–124. [-]

BC23. Dung Bui and Geoffroy Couteau. Improved private set intersection for sets with small entries. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part II*, volume 13941 of *LNCS*, pages 190–220. Springer, Heidelberg, May 2023. [-]

BCG+19a. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. Efficient two-round OT extension and silent non-interactive secure computation. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 291–308. ACM Press, November 2019. [-]

BCG+19b. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 489–518. Springer, Heidelberg, August 2019. [-]

BCG+20. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators from ring-LPN. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 387–416. Springer, Heidelberg, August 2020. [-]

BGI15. Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 337–367. Springer, Heidelberg, April 2015. [-]

CDPP22. Kelong Cong, Debajyoti Das, Jeongeun Park, and Hilder V. L. Pereira. SortingHat: Efficient private decision tree evaluation via homomorphic encryption and transciphering. In Yin et al. [YSCS22], pages 563–577. [-]

CFR23. Anrin Chakraborti, Giulia Fanti, and Michael K. Reiter. Distance-Aware private set intersection. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 319–336, Anaheim, CA, August 2023. USENIX Association. [-]

CHLR18. Hao Chen, Zhicong Huang, Kim Laine, and Peter Rindal. Labeled PSI from fully homomorphic encryption with malicious security. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 1223–1237. ACM Press, October 2018. [-]

CILO22. Wutichai Chongchitmate, Yuval Ishai, Steve Lu, and Rafail Ostrovsky. PSI from ring-OLE. In Yin et al. [YSCS22], pages 531–545. [-]

CMdG+21. Kelong Cong, Radames Cruz Moreno, Mariana Botelho da Gama, Wei Dai, Ilia Iliashenko, Kim Laine, and Michael Rosenberg. Labeled PSI from homomorphic encryption with reduced computation and communication. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 1135–1150. ACM Press, November 2021. [-]

FNP04. Michael J. Freedman, Kobbi Nissim, and Benny Pinkas. Efficient private matching and set intersection. In Christian Cachin and Jan Camenisch, editors, *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 1–19. Springer, Heidelberg, May 2004. [-]

Gal19. Galois, Inc. swanky: A suite of rust libraries for secure computation. `https://github.com/GaloisInc/swanky`, 2019. [-]

GGM24. Gayathri Garimella, Benjamin Goff, and Peihan Miao. Computation efficient structure aware PSI from incremental function secret sharing. Cryptology ePrint Archive, Paper 2024/842, 2024. `https://eprint.iacr.org/2024/842`. [-]

GN19. Satrajit Ghosh and Tobias Nilges. An algebraic approach to maliciously secure private set intersection. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part III*, volume 11478 of *LNCS*, pages 154–185. Springer, Heidelberg, May 2019. [-]

GQL+24. Ying Gao, Lin Qi, Xiang Liu, Yuanchao Luo, and Longxin Wang. Efficient fuzzy private set intersection from fuzzy mapping. In *Advances in Cryptology – ASIACRYPT 2024: 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9–13, 2024, Proceedings, Part VI*, page 36–68, Berlin, Heidelberg, 2024. Springer-Verlag. [-]

GRS22. Gayathri Garimella, Mike Rosulek, and Jaspal Singh. Structure-aware private set intersection, with applications to fuzzy matching. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 323–352. Springer, Heidelberg, August 2022. [-]

GRS23. Gayathri Garimella, Mike Rosulek, and Jaspal Singh. Malicious secure, structure-aware private set intersection. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 577–610. Springer, Heidelberg, August 2023. [-]

GS19. Satrajit Ghosh and Mark Simkin. The communication complexity of threshold private set intersection. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 3–29. Springer, Heidelberg, August 2019. [-]

HFH99. Bernardo A. Huberman, Matt Franklin, and Tad Hogg. Enhancing privacy and trust in electronic communities. In *Proceedings of the 1st ACM Conference on Electronic Commerce*, EC '99, page 78–86, New York, NY, USA, 1999. Association for Computing Machinery. [-]

HOS17. Per A. Hallgren, Claudio Orlandi, and Andrei Sabelfeld. PrivatePool: Privacy-preserving ridesharing. In Boris Köpf and Steve Chong, editors, *CSF 2017 Computer Security Foundations Symposium*, pages 276–291. IEEE Computer Society Press, 2017. [-]

IKN+19. Mihaela Ion, Ben Kreuter, Ahmet Erhan Nergiz, Sarvar Patel, Mariana Raykova, Shobhit Saxena, Karn Seth, David Shanahan, and Moti Yung. On deploying secure computing commercially: Private intersection-sum protocols and their business applications. Cryptology ePrint Archive, Report 2019/723, 2019. `https://eprint.iacr.org/2019/723`. [-]

KRS+19. Daniel Kales, Christian Rechberger, Thomas Schneider, Matthias Senker, and Christian Weinert. Mobile private contact discovery at scale. In Nadia Heninger and Patrick Traynor, editors, *USENIX Security 2019*, pages 1447–1464. USENIX Association, August 2019. [-]

LWYY22. Hanlin Liu, Xiao Wang, Kang Yang, and Yu Yu. The hardness of LPN over any integer ring and field for PCG applications. Cryptology ePrint Archive, Report 2022/712, 2022. `https://eprint.iacr.org/2022/712`. [-]

MTZ03. Y. Minsky, A. Trachtenberg, and R. Zippel. Set reconciliation with nearly optimal communication complexity. *IEEE Transactions on Information Theory*, 49(9):2213–2218, 2003. [-]

PRTY20. Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. PSI from PaXoS: Fast, malicious private set intersection. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 739–767. Springer, Heidelberg, May 2020. [-]

PSSW09. Benny Pinkas, Thomas Schneider, Nigel P. Smart, and Stephen C. Williams. Secure two-party computation is practical. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 250–267. Springer, Heidelberg, December 2009. [-]

RR22. Srinivasan Raghuraman and Peter Rindal. Blazing fast PSI from improved OKVS and subfield VOLE. In Yin et al. [YSCS22], pages 2505–2517. [-]

RS21. Peter Rindal and Phillipp Schoppmann. VOLE-PSI: Fast OPRF and circuit-PSI from vector-OLE. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 901–930. Springer, Heidelberg, October 2021. [-]

UCK+21. Erkam Uzun, Simon P. Chung, Vladimir Kolesnikov, Alexandra Boldyreva, and Wenke Lee. Fuzzy labeled private set intersection with applications to private real-time biometric search. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 911–928. USENIX Association, August 2021. [-]

vBP24. Aron van Baarsen and Sihang Pu. Fuzzy private set intersection with large hyperballs. Cryptology ePrint Archive, Paper 2024/330, 2024. `https://eprint.iacr.org/2024/330`. [-]

WYKW21. Chenkai Weng, Kang Yang, Jonathan Katz, and Xiao Wang. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In *2021 IEEE Symposium on Security and Privacy*, pages 1074–1091. IEEE Computer Society Press, May 2021. [-]

Yao86. Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167. IEEE Computer Society Press, October 1986. [-]
YSCS22. Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors. *ACM CCS 2022*. ACM Press, November 2022. [-]
ZCL21. En Zhang, Jian Chang, and Yu Li. Efficient threshold private set intersection. *IEEE Access*, PP:1–1, 01 2021. [-]

Supplementary Material

A Optimizing for Semi-Honest TLPSI using Padding

A downside in our TLPSI construction is that the receiver needs to perform trial reconstruction to find the final label. Although we can parameterize for a small t, it is still the main computation bottleneck in our protocol. This section presents a technique for efficiently reconstructing the label without trying $\binom{d}{t}$ reconstructions.

The main idea is to add a random padding to sender's input polynomial so that when receiver evaluates her polynomial v_0 she will obtain a padding and a share, instead of only a share. It is important to use a random padding for every element in sender's set since a fixed padding would reveal a partial match even when the threshold is not reached. A consequence of this approach is that we can only support $t = 2$ since receiver cannot distinguish whether there is a match from looking at one random padding.

A.1 Idea

Our initial protocol works as follows where all polynomials (also the vector ring-OLE) are in $\mathbb{F}_q[X]$. receiver has $u_0(X) = \prod_{i=1}^{d}(X - x_i)$, sender has $v_1(X)$ which is degree $2d$ and $v_1(y_i) = f(y_i)$ where $f(0) = \ell$. Finally receiver obtains $v_0 = u_0 \cdot u_1 + v_1$.

The padded version works over $\mathbb{F}_{q'}[X]$ where $q' > q$. Use the same f as before, i.e., in $\mathbb{F}_q[X]$, sender define a set of points $\{(y_i, \mathsf{padding} \| f(y_i))\}_{i\in[d]}$ and interpolate these points to create the degree d polynomial v_1'. Let $v_1 := v_1' \cdot r$ where $r \leftarrow\$\ \mathbb{F}_{q'}[X]$ and $\deg r = d$. receiver still uses $u_0(X) = \prod_{i=1}^{d}(X - x_i)$, except that x_i is casted into the larger field $\mathbb{F}_{q'}$. As before, receiver obtains $v_0 = u_0 \cdot u_1 + v_1 \in \mathbb{F}_{q'}[X]$ where $u_1 \leftarrow\$\ \mathbb{F}_{q'}[X]$ and $\deg u_1 = d$. Evaluating v_0 on an element inside the intersection outputs $v_1(y_i) = \mathsf{padding} \| f(y_i)$. If receiver sees $\mathsf{padding} \| f(y_i)$ and $\mathsf{padding} \| f(y_j)$ for $i \neq j$ then she can reconstruct the label using $\{(y_i, f(y_i)), (y_j, f(y_j))\}$.

A.2 Parameters

The consequence of using the padding is that we must extend the field size from $\mathbb{F}_q$ to $\mathbb{F}_{q'}$, where the padding size is $l_{\mathsf{pad}} = \log_2(|\mathbb{F}_{q'}| - |\mathbb{F}_q|)$ bits. False positives may occur if l_{pad} is small but we can overcome this problem by still sending the hashed label. As a result, there will be no additional false positives. We pick $q = 2^{128}$ and $q' = 2^{192}$ to support $l_{\mathsf{pad}} = 64$. This removes $m \cdot \binom{d}{t}$ reconstruction and hashing but increases the online bandwidth by 50%.